

Criptografía y Seguridad

Código 72.44 – Exámenes Resueltos

Parciales y Recuperatorios con consignas, tips y resoluciones detalladas

[Parcial 2015](#)[Parcial 2016](#)[Parcial 2017](#)[Parcial 2018](#)[Parcial 2023](#)[Parcial 2025-2](#)[Parcial 2026](#)[Recuperatorio 2025](#)

Índice de Contenidos

Recuperatorio 1 – 2025

- Ej. 1 – Protocolo Diffie-Hellman: detalle y seguridad computacional
- Ej. 2 – Por qué el modo ECB no es seguro
- Ej. 3 – Criptosistema que pasa $\text{PrivK}^{\text{CCA}}$
- Ej. 4 – Verdadero o Falso (DES, hash, OTP, Kerckhoff)

Parcial 1 – 2026

- Ej. 1 – Protocolo Diffie-Hellman: tipo, ejemplo numérico, seguridad y problemas
- Ej. 2 – Modo de cifrado en bloque $C_i = E_k(M_i) \oplus C_{i-1}$: validez y comparación
- Ej. 3 – Criptosistema $c = E_{k1}(m \parallel H(k2 \parallel m))$: propiedades
- Ej. 4 – Experimento Expav con $c = (m \oplus k_0) \oplus f(k_1)$
- Ej. 5 – Verdadero o Falso (inyectividad, clave pública, transposición, certificado)

Parcial 1 – 2025-2

- Ej. 1 – Protocolo de autenticación mutua con MAC
- Ej. 2 – Criptoanálisis de Vigenère (texto en castellano)
- Ej. 3 – Modo CBC: validez y comparación con CTR/OFB
- Ej. 4 – Secreto perfecto del cifrado Vigenère generalizado
- Ej. 5 – Verdadero o Falso (MD5, MAC, RSA padding, certificados)

Parcial 1 – 2023

- Ej. 1 – Protocolo TLS-like: tipo, Finished y derivación de clave
- Ej. 2 – CBC: eliminación de bloques C_0 y C_n , prepending
- Ej. 3 – Base64 como sistema de encriptación: confusión, difusión, linealidad
- Ej. 4 – $\text{PrivK}^{\text{CPA}}$ con $r = (a+b)$ determinístico
- Ej. 5 – Verdadero o Falso (Encrypt-and-MAC, hash, DH, PRF invertible)

Parcial 1 – 2018

- Ej. 1 – Protocolo Needham-Schroeder con KDC
- Ej. 2 – Opción múltiple (certificado digital, homofonico, TLS)

- Ej. 3 – CBC: eliminación de bloques C_0 y C_n , prepending
- Ej. 4 – PrivK^{CPA} con $r = (a+b)$ determinístico
- Ej. 5 – Verdadero o Falso

Parcial 1 – 2017

- Ej. 1 – Cilindro de Jefferson: espacio de claves y secreto perfecto
- Ej. 2 – Shamir's No-Key Protocol: objetivo, seguridad y diferencias con DH
- Ej. 3 – Validación de certificado en browser (homebanking)
- Ej. 4 – MAC-Forge con MAC de paridad
- Ej. 5 – Opción múltiple (OFB, CVV, distribución de software)

Parcial 1 – 2016

- Ej. 1 – Transposición por columnas: secreto perfecto formal
- Ej. 2 – Protocolos con hash y firma: propiedades de seguridad
- Ej. 3 – Modo FSM (Fischer Spiffy Mixer): descifrado y propagación de errores
- Ej. 4 – PKI vs KDC: significado y comparación
- Ej. 5 – Opción múltiple (DH exchange, CPA-seguro, CBC-MAC)

Parcial 1 – 2015

- Ej. 1 – Cifrado Hill en Z_2 : espacio de claves, tabla de encriptación, secreto perfecto
- Ej. 2 – MAC no seguro: experimento MAC-Forge
- Ej. 3 – Propagating CBC: descifrado y propagación de errores
- Ej. 4 – Opción múltiple (clave pública, firma RSA)
- Ej. 5 – Ataque de suplantación en protocolo de autenticación

Recuperatorio 1 – 2025

Condición de aprobación: 6 puntos · 4 ejercicios de 2,5 pts c/u

Ejercicio 1 – Diffie-Hellman (2,5 pts)

CONSIGNA

Detallar el protocolo de intercambio de claves de Diffie-Hellman. ¿En qué se basa la seguridad computacional del mismo?

TIP

Mencionar que es un protocolo de *acuerdo* (no envío) de clave. Siempre citar el Problema del Logaritmo Discreto (DLP). Aclarar que es vulnerable a MitM.

TEORÍA

DH permite que dos partes deriven una clave compartida sobre un canal inseguro sin haber compartido secretos previamente. La seguridad se basa en la dificultad del DLP: dado g , p y $g^x \bmod p$, encontrar x es computacionalmente inviable.

RESOLUCIÓN

PROTOCOLO DE DIFFIE-HELLMAN:

Parámetros públicos: primo p , generador g de $\mathbb{Z}_p^*$

1. Gen: A elige $a \leftarrow_R \mathbb{Z}_{p-1}$, calcula $h1 = g^a \bmod p \rightarrow$ envía $h1$ a B
2. B elige $b \leftarrow_R \mathbb{Z}_{p-1}$, calcula $h2 = g^b \bmod p \rightarrow$ envía $h2$ a A
3. A calcula $kA = h2^a = g^{(ab)} \bmod p$
4. B calcula $kB = h1^b = g^{(ab)} \bmod p$
5. Resultado: $kA = kB = g^{(ab)} \bmod p \rightarrow$ clave compartida

SEGURIDAD COMPUTACIONAL:

La seguridad reside en el Problema del Logaritmo Discreto (DLP):
dado g , p y $g^x \bmod p$, es computacionalmente inviable encontrar x .

Un atacante que intercepta $h1 = g^a$ y $h2 = g^b$ no puede calcular $g^{(ab)}$ sin resolver el DLP (Problema Diffie-Hellman Computacional - CDH).

NOTA: No autentifica $\rightarrow$ vulnerable a Man-in-the-Middle (MitM).

Ejercicio 2 – ECB no es seguro (2,5 pts)

CONSIGNA

¿Por qué razón el modo de encadenamiento en bloque ECB no es seguro?

TIP

La respuesta clave: bloques iguales → cifrados iguales. Mencionar el ataque de análisis de patrones y el ejemplo de la imagen del pingüino Tux. ECB no es IND-CPA porque Enc es determinístico.

TEORÍA

ECB (Electronic CodeBook) cifra cada bloque independientemente con la misma clave, sin ningún estado o retroalimentación entre bloques. Para ser IND-CPA seguro se requiere aleatorización.

RESOLUCIÓN

¿POR QUÉ ECB NO ES SEGURO?

En ECB: $C_i = E_k(M_i)$ para cada bloque i , de forma independiente.

PROBLEMA FUNDAMENTAL – Determinismo sin contexto:

Si $M_i = M_j \rightarrow C_i = C_j$ (bloques iguales producen cifrados iguales)

Consecuencias:

1. PATRONES VISIBLES: Un atacante puede inferir estructura del mensaje sin conocer la clave. Ejemplo clásico: imagen del pingüino Tux cifrada en ECB sigue siendo reconocible visualmente.
2. ATAQUE DE REORDENAMIENTO: Un atacante puede reordenar, eliminar o repetir bloques sin que el receptor lo detecte.
3. ATAQUE DE DICCIONARIO: Si el atacante conoce algunos bloques de plaintext (ej: encabezados fijos), puede construir un diccionario $M_i \rightarrow C_i$ y descryptar esos bloques en mensajes futuros.

CONCLUSIÓN: ECB no es IND-CPA seguro porque Enc es determinístico.

Para ser IND-CPA seguro se requiere aleatorización (CBC con IV random, CTR, OFB, etc.).

Ejercicio 3 – Criptosistema que pasa $\text{PrivK}^{\text{CCA}}$ (2,5 pts)

CONSIGNA

Dar un ejemplo de un criptosistema que pase el experimento $\text{PrivK}^{\text{CCA}}_{A,\Pi}$.

TIP

CCA = Chosen Ciphertext Attack. Usar construcción Encrypt-then-MAC (EtM) o mencionar AES-GCM. Lo más importante: explicar *por qué* el MAC impide que el atacante explote el oráculo de descifrado.

TEORÍA

$\text{PrivK}^{\text{CCA}}$: el adversario puede consultar un oráculo de descifrado (excepto sobre el challenge). Un esquema es CCA-seguro si aun así no puede distinguir los mensajes. Requiere tanto confidencialidad como integridad.

RESOLUCIÓN

EJEMPLO: Esquema Encrypt-then-MAC (EtM)

Construcción:

- $\Pi_1 = (\text{Gen}_1, \text{Enc}_1, \text{Dec}_1)$: esquema CPA-seguro (ej: AES-CTR con IV random)
- $\Pi_2 = (\text{Gen}_2, \text{Mac}_2, \text{Vrfy}_2)$: esquema MAC seguro (ej: HMAC-SHA256)

Gen: genera $k_1 \leftarrow_R \{0,1\}^n$, $k_2 \leftarrow_R \{0,1\}^n \rightarrow k = (k_1, k_2)$

$\text{Enc}_k(m)$:

1. $c' = \text{Enc}_{k_1}(m)$ (cifrar el mensaje)
2. $t = \text{Mac}_{k_2}(c')$ (MAC sobre el cifrado)
3. devolver $c = (c', t)$

$\text{Dec}_k(c', t)$:

1. Verificar $\text{Vrfy}_{k_2}(c', t) = 1$, si no $\rightarrow$ rechazar
2. devolver $\text{Dec}_{k_1}(c')$

¿POR QUÉ PASA $\text{PrivK}^{\text{CCA}}$?

- El MAC sobre el cifrado impide que el atacante genere cifrados válidos modificados $\rightarrow$ consultas al oráculo de descifrado son inútiles porque cualquier c modificado falla la verificación del MAC.
- La seguridad CCA se reduce a la seguridad CPA de Π_1 y la seguridad del MAC Π_2 .

Alternativamente: AES-GCM es un esquema AEAD que provee CCA-seguridad directamente.

Ejercicio 4 – Verdadero o Falso (2,5 pts)

CONSIGNA

Verdadero o Falso. Si es falso, corrija la sentencia e identifique el cambio realizado.

- a) Para cualquier sistema que requiera mantener la confidencialidad de un canal, el algoritmo de DES puede usarse sin inconvenientes.
- b) Una función de hash es una buena alternativa para encriptar un mensaje.
- c) One-Time-Pad es un algoritmo seguro contra ataques de texto plano exclusivamente.
- d) El principio de Kerckhoff establece que la seguridad de un criptosistema sólo depende de que el algoritmo de encriptación esté bien protegido.

TIP

En V/F hay que ser muy preciso. DES = 56 bits = inseguro hoy. Hash = unidireccional, no reversible. OTP = secreto perfecto (Shannon) = seguro contra TODO. Kerckhoff = seguridad en la CLAVE, no en el algoritmo.

RESOLUCIÓN

a) FALSO

DES usa claves de 56 bits, hoy considerado inseguro (vulnerable a fuerza bruta, demostrado en 1999 con EFF DES Cracker en menos de 24 hs).

CORRECCIÓN: "DES NO puede usarse sin inconvenientes... Se debe usar AES (128/192/256 bits) en su lugar."

[Cambio: "puede usarse sin inconvenientes" → "NO puede usarse"]

b) FALSO

Las funciones de hash son unidireccionales (no son reversibles), por lo que NO se puede recuperar el mensaje original. No son esquemas de encriptación.

CORRECCIÓN: "Una función de hash NO es una alternativa para encriptar un mensaje, ya que no es invertible. Se usa para integridad, no confidencialidad."

[Cambio: eliminada la afirmación de que es "buena alternativa"]

c) FALSO

OTP tiene secreto perfecto (Shannon), lo que significa que es seguro contra CUALQUIER tipo de ataque, incluso con poder computacional ilimitado.

No es "exclusivamente" contra texto plano.

CORRECCIÓN: "OTP es un algoritmo con secreto perfecto, por lo que es seguro"

contra TODO tipo de ataque (no exclusivamente contra texto plano conocido)."

[Cambio: "exclusivamente" → eliminado; "texto plano" → "todo tipo de ataque"]

d) FALSO

Kerckhoff dice exactamente lo contrario: el algoritmo puede ser público; la seguridad debe residir ÚNICAMENTE en el secreto de la clave.

CORRECCIÓN: "...la seguridad de un criptosistema sólo depende de que LA CLAVE esté bien protegida [no el algoritmo]."

[Cambio: "algoritmo de encriptación" → "clave"]

Parcial 1 – 2026

Condición de aprobación: 6 puntos · 5 ejercicios de 2 pts c/u · Duración: 120 min

Ejercicio 1 – Protocolo Diffie-Hellman (2 pts)

CONSIGNA

Dado el siguiente protocolo donde $A(G, q, g)$ elige un Grupo $G \cong \mathbb{Z}_q$ con raíz primitiva g :

(1.1) $A \rightarrow B: G, q, g$ | (1.2) $A: x \leftarrow \mathbb{Z}_q$ | (1.3) $A: h_1 = g^x$ | (1.4) $A \rightarrow B: h_1$

(1.5) $B: y \leftarrow \mathbb{Z}_q$ | (1.6) $B \rightarrow A: h_2 = g^y$ | (1.7) $A: k_A = h_2^x$ | (1.8) $B: k_B = h_1^y$

- ¿Qué tipo de protocolo sería, qué es lo que intenta construir?
- Mostrar un ejemplo numérico acotado. ¿Qué valores de q son válidos y por qué?
- ¿En qué reside la seguridad computacional?
- Mencionar dos problemas que tiene este protocolo.

TIP

Usar números pequeños en el ejemplo ($p=23$, $g=5$). q debe ser primo para que el grupo tenga orden primo $\rightarrow$ máxima seguridad contra Pohlig-Hellman. Los dos problemas son MitM y falta de autenticación.

RESOLUCIÓN

a) TIPO DE PROTOCOLO:

Es un protocolo de INTERCAMBIO DE CLAVES (Key Agreement) basado en Diffie-Hellman. Construye una clave de sesión compartida entre A y B sobre un canal inseguro sin haber compartido secretos previamente.

b) EJEMPLO NUMÉRICO:

Sea $p=23$ (primo), $g=5$ (generador de $\mathbb{Z}_{23}^*$), $q=22$ (orden del grupo)

A : elige $x=6 \rightarrow h_1 = 5^6 \bmod 23 = 8 \rightarrow$ envía 8 a B

B : elige $y=15 \rightarrow h_2 = 5^{15} \bmod 23 = 19 \rightarrow$ envía 19 a A

$A: k_A = 19^6 \bmod 23 = 2$

$B: k_B = 8^{15} \bmod 23 = 2 \quad \checkmark \quad k_A = k_B = 2$ (clave compartida)

VALORES DE q VÁLIDOS:

q debe ser PRIMO (o tener un factor primo grande). Si q es primo, el grupo cíclico tiene orden primo $\rightarrow$ no existen subgrupos pequeños que puedan ser explotados por el algoritmo de Pohlig-Hellman para factorizar el problema del logaritmo discreto en partes más simples.

c) SEGURIDAD COMPUTACIONAL:

Reside en el Problema del Logaritmo Discreto (DLP):

dado g , p y $g^x \bmod p \rightarrow$ encontrar x es computacionalmente inviable

para p suficientemente grande (≥ 2048 bits actualmente).

El atacante ve $h_1 = g^x$ y $h_2 = g^y$ pero no puede calcular $g^{(xy)}$ sin resolver el CDH (Computational Diffie-Hellman Problem).

d) DOS PROBLEMAS DEL PROTOCOLO:

1. NO AUTENTICACIÓN → Vulnerable a ataque Man-in-the-Middle (MitM):
un atacante C puede interceptar h_1 y h_2 , y establecer claves separadas con A y con B, pasando mensajes entre ellos sin que ninguno lo detecte.
2. NO PROVEE AUTENTICACIÓN DE IDENTIDAD: A y B no pueden verificar que están hablando con quien creen. Cualquier entidad puede iniciar el protocolo como si fuera A o B.

Ejercicio 2 – Modo de cifrado $C_i = E_k(M_i) \oplus C_{i-1}$ (2 pts)

CONSIGNA

Consideren el siguiente sistema de encriptación en bloque para mensajes $M_1M_2\dots M_n$ que generan cifrados $C_0C_1C_2\dots C_n$:

$$C_0 = IV \mid C_i = E_k(M_i) \oplus C_{i-1}, i=1,2,\dots$$

- a) ¿Es este un esquema de cifrado en bloque válido? Explicar.
b) Comparar la confidencialidad y la tolerancia a errores contra CBC, CTR y OFB.

TIP

Este NO es el CBC estándar. CBC estándar: $C_i = E_k(M_i \oplus C_{i-1})$. Este esquema: primero cifra, luego XOR. Verificar que la función de descifrado esté bien definida. Las propiedades de propagación de errores cambian.

RESOLUCIÓN

a) ¿ES VÁLIDO?

SÍ, es un esquema válido. Existe función de descifrado inversa bien definida:

$$\text{De } C_i = E_k(M_i) \oplus C_{i-1}:$$

$$\rightarrow E_k(M_i) = C_i \oplus C_{i-1}$$

$$\rightarrow M_i = D_k(C_i \oplus C_{i-1}) \quad \leftarrow \text{función de descifrado}$$

$$\text{Verificación: } D_k(C_i \oplus C_{i-1}) = D_k(E_k(M_i) \oplus C_{i-1} \oplus C_{i-1}) = D_k(E_k(M_i)) = M_i \quad \checkmark$$

Con IV aleatorio y E_k como PRP, es IND-CPA seguro.

b) COMPARACIÓN:

CONFIDENCIALIDAD:

- Este esquema: similar a CBC, IND-CPA seguro con IV aleatorio. Cada C_i depende de todos los bloques anteriores (encadenamiento).
- CBC estándar: IND-CPA seguro con IV aleatorio. Equivalente.
- CTR: IND-CPA seguro con nonce único. Totalmente paralelizable.
- OFB: IND-CPA seguro. Keystream independiente del mensaje.

TOLERANCIA A ERRORES DE TRANSMISIÓN (error en bloque C_i):

Modo	Efecto de un error en C_i

ESTE esq.	$M_i = D_k(C_i \oplus C_{i-1}) \rightarrow M_i$ corrupto (todos bits)
	$M_{i+1} = D_k(C_{i+1} \oplus C_i) \rightarrow M_{i+1}$ corrupto (todos)
	M_{i+2} en adelante: OK (C_i ya no interviene)
CBC std	M_i corrupto (todos bits) + 1 bit de M_{i+1} corrupto
	M_{i+2} en adelante: OK
CTR	Solo afecta el bit correspondiente de M_i .
	No se propaga. Mejor tolerancia.
OFB	Solo afecta M_i (bits correspondientes).
	No se propaga. Mejor tolerancia.

DIFERENCIA CLAVE vs CBC estándar:

En este esquema, un error en C_i afecta COMPLETAMENTE a M_i y COMPLETAMENTE a M_{i+1} (ya que C_i aparece en el descifrado de ambos).

En CBC estándar, solo 1 bit de M_{i+1} queda corrupto.

Ejercicio 3 – Criptosistema $c = Ek_1(m \parallel H(k_2 \parallel m))$ (2 pts)

CONSIGNA

Dado el criptosistema $c = Ek_1(m \parallel H(k_2 \parallel m))$, donde m es mensaje de tamaño fijo, H es función de hash criptográfica, $Ek(\cdot)$ es primitiva de encriptación simétrica, k_1 y k_2 son claves compartidas entre Bob y Alice.

- Detallar paso a paso lo que debe hacer el receptor al recibir c .
- ¿Puede un atacante modificar el mensaje? Explicar la integridad.
- ¿Provee algún esquema de autenticación? Explicar.
- ¿Provee algún mecanismo de no-repudio? Explicar.

TIP

Analizar cada propiedad por separado. $H(k_2 \parallel m)$ es un MAC (secret-prefix). k_1 da confidencialidad. Autenticación SÍ (entre partes que comparten k_2). No repudio NO (requiere asimetría).

RESOLUCIÓN

ESQUEMA: $c = Ek_1(m \parallel H(k_2 \parallel m))$

a) PASOS DEL RECEPTOR:

- Descifrar: $(m', h') = Dk_1(c)$ usando k_1
- Recomputar: $h_check = H(k_2 \parallel m')$ usando k_2
- Comparar: si $h' == h_check \rightarrow$ mensaje íntegro y auténtico
- Si verificación OK $\rightarrow$ aceptar m'
Si no $\rightarrow$ rechazar (el mensaje fue alterado o no es auténtico)

b) INTEGRIDAD:

SÍ provee integridad. $H(k_2 \parallel m)$ actúa como un MAC (construcción secret-prefix).

Un atacante que no conoce k_2 no puede:

- Modificar m' sin que cambie $H(k_2 \parallel m')$
 - Forjar un nuevo par $(m^*, H(k_2 \parallel m^*))$ válido sin conocer k_2
- $\rightarrow$ Cualquier modificación es detectada en el paso 3.

NOTA: La construcción $H(k_2 \parallel m)$ es potencialmente vulnerable a length-extension attack en hashes tipo SHA-256/MD5. La construcción recomendada es $HMAC = H(k_2 \oplus opad \parallel H(k_2 \oplus ipad \parallel m))$.

c) AUTENTICACIÓN:

SÍ, provee autenticación de origen SIMÉTRICA entre las partes que

comparten k_2 . Solo quien conoce k_2 puede generar un MAC válido.

Limitación: es autenticación simétrica → ambas partes comparten k_2 , no se puede distinguir quién originó el mensaje ante un tercero.

d) NO REPUDIO:

NO provee no repudio. El no repudio requiere firma digital asimétrica (clave privada que solo posee el emisor). Aquí tanto A como B conocen k_2 , por lo que B no puede probar ante un tercero que el mensaje fue generado por A y no por él mismo.

Ejercicio 4 – Experimento Expav(A,n) (2 pts)

CONSIGNA

Dado el criptosistema Expav(A,n): $c = E_k(m) = (m \oplus k_0) \oplus f(k_1)$, con $f(\cdot)$ la función identidad ($f(0)=0$, $f(1)=1$), $k = k_0k_1 \in \{0,1\}^2$ uniformemente distribuidas, $m, c \in \{0,1\}$, $\Pr[m=0]=0,9$ y $\Pr[m=1]=0,1$. Verificar si un atacante tiene éxito en un ataque de texto cifrado.

TIP

Simplificar primero: f es identidad $\rightarrow c = m \oplus k_0 \oplus k_1$. Sea $K = k_0 \oplus k_1$, que es uniforme en $\{0,1\}$. Esto es un OTP de 1 bit. Pero la distribución de m NO es uniforme $\rightarrow$ el atacante puede explotar el prior.

RESOLUCIÓN

SIMPLIFICACIÓN:

f es la función identidad: $f(k_1) = k_1$

$\rightarrow c = m \oplus k_0 \oplus k_1$

Sea $K = k_0 \oplus k_1$. Como k_0, k_1 son uniformes e independientes en $\{0,1\}$:

$\Pr[K=0] = \Pr[k_0=k_1] = 1/2$ y $\Pr[K=1] = 1/2 \rightarrow K$ es uniforme en $\{0,1\}$

$\rightarrow c = m \oplus K$ (equivalente a OTP de 1 bit)

ANÁLISIS DEL ATAQUE:

Dado c , el atacante calcula:

$$\begin{aligned} \Pr[m=0|c=0] &= \Pr[c=0|m=0] \cdot \Pr[m=0] / \Pr[c=0] \\ &= (1/2 \cdot 0,9) / (1/2) = 0,9 \end{aligned}$$

$$\Pr[m=1|c=0] = 0,1 \quad (\text{complemento})$$

Similarmente:

$$\Pr[m=0|c=1] = 0,9 \quad \text{y} \quad \Pr[m=1|c=1] = 0,1$$

ESTRATEGIA ÓPTIMA DEL ATACANTE:

Sin importar el valor de c , el atacante SIEMPRE adivina $m=0$.

$$\Pr[\text{éxito}] = 0,9$$

CONCLUSIÓN:

El esquema NO tiene secreto perfecto a pesar de ser tipo OTP, porque la distribución de mensajes NO es uniforme. El atacante gana con probabilidad $0,9 > 1/2$, obteniendo ventaja:

$$\text{Adv} = |\Pr[A \text{ gana}] - 1/2| = |0,9 - 0,5| = 0,4$$

El sistema NO es seguro bajo distribución no uniforme de mensajes.

Ejercicio 5 – Verdadero o Falso (2 pts)

CONSIGNA

- a) Cualquier función de encriptación simétrica tiene que ser inyectiva.
- b) Los sistemas de encriptación basados en clave pública utilizan una de las claves para encriptar/desencriptar y la otra para firmar.
- c) Para sistemas de cifrado con un código binario de tres bits, el número de cifrados de transposición diferentes es mayor que el número de cifrados de sustitución.
- d) Un certificado público contiene la clave privada de la entidad certificante.

TIP

Inyectividad: sin ella Dec sería ambiguo. Clave pública: pública encripta/verifica, privada descifra/firma (no son roles separados). Sustitución sobre $\{0,1\}^3$: $8!$ permutaciones vs $3!$ transposiciones. Certificado: contiene clave pública del sujeto.

RESOLUCIÓN

a) VERDADERO

Enc_k debe ser inyectiva para que Dec_k sea bien definida (biyectiva fijo k).

Si no fuera inyectiva, dos mensajes distintos darían el mismo cifrado y no podría descifrarse unívocamente.

b) FALSO

En sistemas de clave pública la clave PÚBLICA encripta (y verifica firmas) y la clave PRIVADA descifra (y firma). No es que una clave haga Enc/Dec y la otra firme: son los dos roles del MISMO par de claves.

CORRECCIÓN: "...utilizan la clave pública para encriptar y verificar firmas,

y la clave privada para descifrar y firmar."

[Cambio: descripción de roles de cada clave]

c) FALSO

Para código binario de 3 bits (mensajes de 3 bits, 8 símbolos posibles):

- Cifrados de SUSTITUCIÓN: permutaciones de 8 símbolos $\rightarrow 8! = 40.320$

- Cifrados de TRANSPOSICIÓN: permutaciones de 3 posiciones $\rightarrow 3! = 6$

Hay MUCHOS más cifrados de sustitución que de transposición.

CORRECCIÓN: "...el número de cifrados de SUSTITUCIÓN es MAYOR que el de transposición." [Cambio: invertir la relación]

d) FALSO

Un certificado digital (X.509) contiene la clave PÚBLICA del sujeto certificado y está firmado con la clave PRIVADA de la CA.

Nunca contiene claves privadas.

CORRECCIÓN: "Un certificado público contiene la clave PÚBLICA DEL SUJETO CERTIFICADO [no la clave privada de la CA]."

[Cambio: "clave privada de la CA" → "clave pública del sujeto"]

Parcial 1 – 2025-2

Condición de aprobación: 6 puntos · 5 ejercicios de 2 pts c/u · Duración: 120 min

Ejercicio 1 – Protocolo de autenticación con MAC (2 pts)

CONSIGNA

Dado el protocolo donde A y B comparten claves simétricas K y K', y $h'K'(\cdot)$ es una función MAC diferente de $hK(\cdot)$:

(1.1) $A \rightarrow B: rA$ | (1.2) $B \rightarrow A: (B, A, rA, rB), hK(B, A, rA, rB, K')$ | (1.3) $A \rightarrow B: (A, rB), hK(A, rB, K')$

(1.4) $A: W = h'K'(rB)$ | (1.5) $B: W = h'K'(rB)$

- ¿Qué tipo de protocolo sería? ¿Qué intenta construir?
- ¿Qué les permiten hacer a A y B los mensajes cruzados 1.2 y 1.3?
- ¿Es susceptible a un ataque MitM? Justificar.

TIP

rA y rB son nonces (números random). El protocolo termina con ambos derivando la misma W → es autenticación + establecimiento de clave. Los MACs con K impiden que un tercero sin K pueda interferir.

RESOLUCIÓN

a) TIPO DE PROTOCOLO:

Es un protocolo de AUTENTICACIÓN MUTUA con ESTABLECIMIENTO DE CLAVE DE SESIÓN.

Construye: (1) autenticación de A y B entre sí, (2) una clave de sesión W derivada de K' y rB, compartida al final de forma segura.

b) MENSAJES 1.2 Y 1.3 – AUTENTICACIÓN MUTUA:

Mensaje 1.2 ($B \rightarrow A$): B se autentica ante A.

- Incluye rA : prueba que B recibió el mensaje de A (frescura).
- Propone rB y K' . El MAC $hK(B, A, rA, rB, K')$ garantiza que B conoce K y que ningún campo fue alterado en tránsito.

Mensaje 1.3 ($A \rightarrow B$): A se autentica ante B.

- Incluye rB : prueba que A recibió el mensaje de B (frescura).
- El MAC $hK(A, rB, K')$ garantiza que A conoce K.

JUNTOS logran AUTENTICACIÓN MUTUA: ninguna parte puede suplantar a la otra sin conocer K. Los nonces garantizan frescura y previenen replay.

c) ¿VULNERABLE A MitM?

NO, el protocolo NO es vulnerable a MitM, porque:

- Los MACs $hK(\cdot)$ dependen de K, que solo A y B conocen.
- Un atacante C que intercepta mensajes no puede forjar MACs válidos

sin K , ni puede modificar los campos sin invalidar el MAC.

- Los nonces r_A y r_B (distintos en cada sesión) previenen ataques de replay.
- La inclusión de K' en el MAC del paso 1.2 vincula la clave de sesión derivada W al handshake, evitando sustitución de clave.

CONCLUSIÓN: El protocolo es seguro contra MitM gracias a los MACs con K .

Ejercicio 2 – Criptoanálisis de Vigenère (2 pts)

CONSIGNA

El siguiente texto fue encontrado en una botella en la guerra de los Roses:

"GWAOTESFENITLAGEUGEDRVPHJVCDFDR"

Se sabe que fue encriptado con clave y estaba en castellano (alfabeto de 26 letras).

- Detallar cómo sería el abordaje para criptoanalizarlo.
- Intentar encontrar la clave y el mensaje.

TIP

Vigenère: $ci = (mi + ki) \bmod 26$. El método es: (1) Kasiski/IC para encontrar longitud de clave, (2) dividir el texto en grupos por posición mod l , (3) análisis de frecuencia en cada grupo. Con solo 30 caracteres el análisis es limitado; mostrar el método vale más que el resultado.

RESOLUCIÓN

TEXTO: "GWAOTESFENITLAGEUGEDRVPHJVCDFDR" (30 caracteres)

a) ABORDAJE DE CRIPTOANÁLISIS:

PASO 1 – Test de Kasiski:

Buscar substrings repetidos en el texto cifrado y calcular el MCD de las distancias entre repeticiones → estima la longitud de la clave l . (Con 30 caracteres es difícil encontrar repeticiones significativas.)

PASO 2 – Índice de Coincidencia (IC):

$$IC = \frac{\sum (n_i \cdot (n_i - 1))}{N \cdot (N - 1)}$$

- Texto en castellano natural: $IC \approx 0.074$

- Texto aleatorio (clave larga): $IC \approx 0.038$

Probar agrupar letras por posición mod l ($l=1,2,3,4,\dots$) y calcular el IC de cada subgrupo. La l correcta dará $IC \approx 0.074$ en cada grupo.

PASO 3 – Análisis de frecuencias por subgrupo:

Para cada posición i ($i=0..l-1$): tomar las letras en posiciones $i, i+l, i+2l, \dots$ y hacer análisis de frecuencia.

La letra más frecuente debería corresponder a 'E' (13%) o 'A' (13%) en castellano. La diferencia da $ki = (ci_{\text{más frecuente}} - E_{\text{ascii}}) \bmod 26$.

b) INTENTO DE DESCIFRADO:

Frecuencias en el texto (30 chars):

E:3, D:3, G:3, R:2, A:2, W:1, O:1, S:1, F:1, N:1, I:1, T:1,

L:1, U:1, V:1, P:1, H:1, J:1, C:1

Letras más frecuentes: E, D, G → candidatos a 'E' o 'A' en castellano.

Probando clave de longitud $l=3$:

- Grupo 0 (pos 0,3,6,...): G,O,F,I,L,G,G,R,H,C → G más frecuente
Si $G \rightarrow E$: $k_0 = (G-E) = (6-4) = 2$
- Grupo 1 (pos 1,4,7,...): W,E,E,T,A,E,E,V,J,D → E más frecuente
Si $E \rightarrow E$: $k_1 = 0$
- Grupo 2 (pos 2,5,8,...): A,S,N,L,G,U,D,P,V,R → varios → difícil

Con $k=(2,0,?)$, descifrar posiciones 0,3,6,...: $G-2=E$, $O-2=M$, $F-2=D...$
→ "EMD..." no parece castellano claro.

NOTA: Con solo 30 caracteres el análisis estadístico es muy limitado.
En el examen lo importante es demostrar el MÉTODO correctamente.

Ejercicio 3 – Modo CBC: validez y comparación (2 pts)

CONSIGNA

Consideren el siguiente sistema de encriptación en bloque para mensajes $M_1M_2\dots M_n$ que generan cifrados $C_0C_1C_2\dots C_n$:

$$C_0 = IV \mid C_i = E_k(C_{i-1} \oplus M_i), i=1,2,\dots$$

- a) ¿Es este un esquema de cifrado en bloque válido? Explicar y eventualmente corregirlo.
b) Comparar la confidencialidad y la tolerancia a errores contra CBC, CTR y OFB.

TIP

Este ES el CBC estándar exactamente. Confirmar validez y mostrar la función de descifrado. En la comparación, este esquema ES CBC, por lo que la comparación es con sí mismo — aclarar que son equivalentes y comparar con los otros dos.

RESOLUCIÓN

ESQUEMA: $C_0=IV, C_i = E_k(C_{i-1} \oplus M_i)$

a) ¿ES VÁLIDO?

SÍ. Este esquema ES el modo CBC estándar (Cipher Block Chaining).
Es válido porque existe función de descifrado bien definida:

$$\text{Dec: } M_i = D_k(C_i) \oplus C_{i-1}$$

Verificación:

$$D_k(C_i) \oplus C_{i-1} = D_k(E_k(C_{i-1} \oplus M_i)) \oplus C_{i-1} = (C_{i-1} \oplus M_i) \oplus C_{i-1} = M_i \quad \checkmark$$

Con IV aleatorio y E_k como PRP (pseudorandom permutation), es IND-CPA seguro.

No requiere corrección.

b) COMPARACIÓN:

CONFIDENCIALIDAD:

- Este esquema (=CBC): IND-CPA seguro con IV aleatorio e imprevisible.
No es IND-CCA (sin MAC adicional).
- CTR: IND-CPA seguro con nonce único. Genera keystream paralelo.
Más eficiente para procesamiento en paralelo.
- OFB: IND-CPA seguro. Keystream independiente del mensaje (útil para canales con ruido donde se puede precomputar el keystream).

TOLERANCIA A ERRORES (error en bloque C_i):

Modo	Efecto de 1 bit erróneo en C_i
CBC (este)	$M_i = D_k(C_i^*) \oplus C_{i-1} \rightarrow M_i$ completamente corrupto $M_{i+1} = D_k(C_{i+1}) \oplus C_i^* \rightarrow$ 1 bit corrupto en M_{i+1} M_{i+2} en adelante: completamente OK
CTR	Solo el/los bit(s) correspondientes de M_i . No se propaga. Mejor tolerancia a errores.
OFB	Solo los bits correspondientes de M_i . No se propaga. Igual que CTR.

CTR y OFB son más tolerantes a errores que CBC.

CBC tiene auto-sincronización parcial (se recupera en 2 bloques).

Ejercicio 4 – Secreto Perfecto del Cifrado Vigenère Generalizado (2 pts)

CONSIGNA

Se define un criptosistema $\Pi=(\text{Gen},\text{Enc},\text{Dec})$ con alfabeto de 26 letras, $K = k_1k_2\dots k_l$ con $k_i \in \{0,\dots,25\}$, mensajes $M = m_1m_2\dots m_n$ con $m_i \in \{0,\dots,25\}$. La encriptación: $c_j = (m_j + k((j-1) \bmod l) + 1) \bmod 26$. Demostrar si este sistema tiene secreto perfecto y bajo qué condiciones sobre los parámetros.

TIP

Este es el cifrado de Vigenère. La condición clave es $l \geq n$ (clave tan larga como el mensaje). Cuando $l = n$ es equivalente a OTP $\rightarrow$ secreto perfecto. Cuando $l < n$ la clave se repite $\rightarrow$ filtra información.

RESOLUCIÓN

SISTEMA: $c_j = (m_j + k((j-1) \bmod l) + 1) \bmod 26$ (Cifrado de Vigenère)

CASO 1: $l \geq n$ (clave tan larga como el mensaje)

Para cada posición j ($1 \leq j \leq n$), la subclave k_j es uniforme en $\{0..25\}$ e independiente de las demás (asumiendo Gen elige k uniformemente).

Dado $c_j = m_j + k_j \bmod 26$:

Para cualquier m_j fijo y cualquier $c_j \in \{0..25\}$:

$$\Pr[C_j=c_j \mid M_j=m_j] = \Pr[K_j = (c_j - m_j) \bmod 26] = 1/26 = \Pr[C_j=c_j]$$

Como las posiciones son independientes y cada posición satisface la condición de secreto perfecto individualmente:

$$\Pr[C=c \mid M=m] = \Pr[C=c] \text{ para todo } m, c \rightarrow \text{SECRETO PERFECTO } \checkmark$$

Cuando $l=n$, cada subclave se usa exactamente una vez $\rightarrow$ equivalente a OTP.

CASO 2: $l < n$ (clave más corta que el mensaje, la clave se repite)

Las posiciones j y $j+1$ usan la MISMA subclave $k((j-1) \bmod l) + 1$.

$$\text{Entonces: } c_j - m_j \equiv c_{j+1} - m_{j+1} \pmod{26}$$

$$\text{Un atacante puede calcular: } c_j - c_{j+1} \equiv m_j - m_{j+1} \pmod{26}$$

$\rightarrow$ Esto FILTRA información sobre la diferencia de caracteres del plaintext.

$\rightarrow$ NO tiene secreto perfecto. (Ejemplo: índice de coincidencia de Kasiski)

CONCLUSIÓN:

El sistema tiene secreto perfecto SI Y SOLO SI:

1. $l \geq n$ (longitud de clave $\geq$ longitud del mensaje)
2. La clave k se elige uniformemente de $\{0, \dots, 25\}^l$
3. La clave se usa una única vez (no se reutiliza)

Cuando $l = n$: idéntico a One-Time Pad $\rightarrow$ secreto perfecto garantizado.

Cuando $l < n$: sistema vulnerable al análisis de Kasiski/IC $\rightarrow$ inseguro.

Ejercicio 5 – Verdadero o Falso (2 pts)

CONSIGNA

- a) MD5 es un criptosistema de encriptación asimétrico que no debe ser utilizado porque usa una longitud de clave de 128 bits.
- b) Un protocolo de autenticación basado únicamente en un MAC simétrico provee confidencialidad, integridad y no repudio entre las partes.
- c) El uso de padding aleatorio en la implementación del algoritmo de clave pública de RSA es para que el algoritmo sea seguro a ataque de textos cifrados elegidos.
- d) Un certificado digital emitido por una autoridad certificante contiene siempre la clave pública de la CA.

TIP

MD5 es función de hash (no cifrado), falla por colisiones (no por longitud). MAC simétrico no da no repudio ni confidencialidad. RSA-OAEP → CCA-seguridad. Certificado X.509 contiene clave pública del sujeto, firmado por CA.

RESOLUCIÓN

a) FALSO (dos errores)

MD5 es una FUNCIÓN DE HASH (no un criptosistema de encriptación) y no es asimétrica (no usa par de claves). No debe usarse porque se han encontrado COLISIONES (no por longitud del digest).

CORRECCIÓN: "MD5 es una función de HASH criptográfica que no debe utilizarse porque se han encontrado colisiones."

[Cambios: "encriptación asimétrico" → "hash"; "longitud de clave" → "colisiones"]

b) FALSO

Un MAC simétrico provee INTEGRIDAD y AUTENTICACIÓN entre las partes.

NO provee:

- Confidencialidad (el MAC no cifra el mensaje)
- No repudio (ambas partes comparten k → cualquiera puede generar el MAC, no se puede probar ante un tercero quién lo generó)

CORRECCIÓN: "...provee integridad y autenticación, pero NO confidencialidad

ni no repudio."

[Cambio: eliminar "confidencialidad" y "no repudio"]

c) VERDADERO

RSA sin padding es determinístico → no es IND-CPA (mismo m siempre da

mismo c).

El padding aleatorio (OAEP – Optimal Asymmetric Encryption Padding) hace que

RSA sea probabilístico y provee seguridad IND-CCA2. ✓

d) FALSO

Un certificado digital X.509 contiene la clave PÚBLICA DEL SUJETO (entidad certificada). Está FIRMADO por la CA con su clave privada, pero no contiene

la clave pública de la CA (esa está en el certificado de la CA, que es independiente).

CORRECCIÓN: "...contiene siempre la clave pública DEL SUJETO CERTIFICADO [no de la CA]."

[Cambio: "de la CA" → "del sujeto certificado"]

Parcial 1 – 2023

Condición de aprobación: 5 puntos · 5 ejercicios de 2 pts c/u · Duración: 90 min

Ejercicio 1 – Protocolo TLS-like (2 pts)

CONSIGNA

Dado el protocolo (similar a TLS 1.2):

(1.1) $C \rightarrow S: C, C\#, NC$ | (1.2) $C \leftarrow S: S, S\#, NS, \text{Cert}(S, \text{Sgn}_{KS}(S))$

(1.3) $C \rightarrow S: \text{EK}_0(KS), N$ | $k_1 = H(K_0, NC, NS)$ | (1.4) $C \rightarrow S: \text{EKcs}(\text{finished}, \text{MAC}_{k_1}(\text{timestamp}))$ | $Kcs = H(N, k_1)$

(1.5) $C \leftarrow S: \text{EKcs}(\text{finished}, \text{MAC}_{k_1}(\text{timestamp}))$ | (1.6/1.7) datos cifrados con Kcs

a) ¿Qué tipo de protocolo sería, qué intenta construir?

b) ¿Cuál es el propósito de los mensajes (1.4) y (1.5)?

c) ¿Por qué se deriva Kcs y no se usa K0?

TIP

Este es esencialmente TLS 1.2. Los mensajes Finished verifican que el handshake fue íntegro. K_0 no se usa directamente para evitar reutilización y garantizar frescura por sesión.

RESOLUCIÓN

a) TIPO: Es un protocolo de ESTABLECIMIENTO DE SESIÓN SEGURA (TLS-like).
Construye: autenticación del servidor (vía certificado + firma),
acuerdo de clave de sesión Kcs, confidencialidad e integridad del canal.

b) PROPÓSITO DE MENSAJES (1.4) Y (1.5) – "Finished":

Son mensajes de CONFIRMACIÓN / VERIFICACIÓN DEL HANDSHAKE.

- El cliente envía (1.4) para probar que derivó Kcs correctamente y que el handshake que recibió es íntegro (MAC sobre timestamp).
- El servidor responde (1.5) para confirmar que también derivó la misma Kcs y que el handshake que él vio es idéntico al del cliente.

Si alguno falla la verificación → la conexión se aborta.

Esto garantiza que ambos lados ven el mismo handshake y tienen la misma clave, previniendo ataques de truncación y downgrade.

c) ¿POR QUÉ DERIVAR Kcs Y NO USAR K_0 DIRECTAMENTE?

1. FRESCURA POR SESIÓN: $Kcs = H(N, k_1) = H(N, H(K_0, NC, NS))$.

Los nonces NC y NS son distintos en cada sesión → Kcs única por sesión, aunque K_0 sea el mismo pre-master secret.

2. SEPARACIÓN DE ROLES: K_0 es el secreto pre-master (confidencial, usado solo para derivar claves). Kcs es la clave operativa de sesión. Esta separación limita el daño si Kcs se compromete.

3. FORWARD SECRECY PARCIAL: Si K_0 se revela a posteriori, no permite descifrar sesiones pasadas porque K_{cs} depende también de nonces efímeros (NC, NS) que no se guardan.

Ejercicio 2 – CBC: eliminación de bloques y prepending (2 pts)

CONSIGNA

En CBC: $C_0 = IV$, $C_k = E_k(M_k \oplus C_{k-1})$.

- ¿Cómo se ve afectada la descrición si el primer bloque C_0 es eliminado?
- ¿Cómo se ve afectada si el último bloque C_n es eliminado?
- ¿Cómo puede un usuario legítimo agregar un bloque M_0 específico al principio del mensaje original?

TIP

Recordar: $M_k = D_k(C_k) \oplus C_{k-1}$. C_0 es el IV $\rightarrow$ su eliminación solo afecta M_1 . C_n eliminado $\rightarrow M_n$ se pierde, el resto OK. Para prepend: calcular $C_{-1} = D_k(C_0) \oplus M_0$.

RESOLUCIÓN

Descifrado CBC: $M_k = D_k(C_k) \oplus C_{k-1}$

a) SE ELIMINA C_0 (IV):

$M_1 = D_k(C_1) \oplus C_0 \rightarrow$ sin C_0 , el receptor usa un valor incorrecto como IV.

$\rightarrow M_1$ se descifra INCORRECTAMENTE (completamente corrupto).

$M_2, M_3, \dots, M_n$: $M_k = D_k(C_k) \oplus C_{k-1}$ (con $k \geq 2$, C_{k-1} está disponible)

$\rightarrow M_2 \dots M_n$ se descifran CORRECTAMENTE.

IMPACTO: Solo M_1 corrupto. El resto del mensaje se recupera.

b) SE ELIMINA C_n (último bloque):

$M_1 \dots M_{n-1}$: descifrado normal, completamente correcto (no dependen de C_n).

$M_n = D_k(C_n) \oplus C_{n-1} \rightarrow$ sin C_n , M_n no puede calcularse.

$\rightarrow M_n$ simplemente se PIERDE.

IMPACTO: Solo M_n se pierde. El resto del mensaje está intacto.

c) AGREGAR M_0 ESPECÍFICO AL PRINCIPIO:

El usuario quiere que, al descifrar, el primer bloque sea M_0 .

Se necesita un nuevo bloque C_{-1} tal que:

$$M_0 = D_k(C_0) \oplus C_{-1}$$

$$\Rightarrow C_{-1} = D_k(C_0) \oplus M_0$$

El nuevo texto cifrado a enviar: $C_{-1}, C_0, C_1, \dots, C_n$

Al descifrar:

$$M_0 = D_k(C_0) \oplus C_{-1} = D_k(C_0) \oplus D_k(C_0) \oplus M_0 = M_0 \quad \checkmark$$

$$M_1 = D_k(C_1) \oplus C_0 \quad (\text{sin cambios}) \quad \checkmark$$

NOTA: Esto requiere que el usuario legítimo conozca D_k (clave de descifrado)

y el valor de C_0 . Demuestra que CBC sin MAC es maleable.

Ejercicio 3 – Base64 como sistema de encriptación (2 pts)

CONSIGNA

El banco de Estander usa base64 como sistema de encriptación simétrica.

- ¿Puede considerarse un sistema de encriptación válido?
- El CSO dice que su sistema ofrece confusión y difusión. ¿Qué significan?
- El CSO insiste en que el sistema no es lineal. ¿Qué significa que un criptosistema sea no lineal?

TIP

Base64 es encoding, no encriptación. No tiene clave → no hay secreto. Confusión y difusión son principios de Shannon. No linealidad = $E_k(m_1 \oplus m_2) \neq E_k(m_1) \oplus E_k(m_2)$. OTP es lineal.

RESOLUCIÓN

a) ¿ES BASE64 UN SISTEMA DE ENCRIPCIÓN VÁLIDO?

NO. Base64 es un esquema de CODIFICACIÓN, no de encriptación:

- No tiene CLAVE: no hay secreto que mantener
- Es completamente REVERSIBLE y PÚBLICO (cualquiera puede decodificar)
- No provee confidencialidad, integridad ni ninguna propiedad criptográfica

- Es una representación de datos en ASCII (3 bytes → 4 caracteres ASCII)

CONCLUSIÓN: No cumple ningún requisito de un criptosistema (Gen, Enc, Dec con k).

b) CONFUSIÓN Y DIFUSIÓN (principios de Shannon, 1949):

CONFUSIÓN: La relación entre la clave y el texto cifrado debe ser lo más compleja e impredecible posible. Cada bit del cifrado debe depender de muchos

bits de la clave de manera no lineal.

→ Dificulta el análisis de la clave a partir del cifrado.

→ En AES: implementada por los S-boxes (sustitución no lineal).

DIFUSIÓN: Cada bit del plaintext debe afectar a muchos bits del cifrado (y viceversa). Cambiar 1 bit en el mensaje debe cambiar ~50% de los bits del cifrado (efecto avalancha).

→ Dificulta el análisis estadístico.

→ En AES: implementada por ShiftRows y MixColumns.

Base64 tiene CERO confusión (no hay clave) y CERO difusión (cada grupo de 3 bytes se codifica independientemente).

c) NO LINEALIDAD:

Un criptosistema es NO LINEAL si la función de cifrado NO satisface:

$$E_k(m_1 \oplus m_2) \neq E_k(m_1) \oplus E_k(m_2)$$

La no linealidad es esencial para seguridad. Sin ella, un atacante puede resolver sistemas de ecuaciones lineales para encontrar la clave.

EJEMPLO DE CRIPTOSISTEMA LINEAL: OTP ($c = m \oplus k$):

$$E_k(m_1 \oplus m_2) = (m_1 \oplus m_2) \oplus k \text{ y } E_k(m_1) \oplus E_k(m_2) = (m_1 \oplus k) \oplus (m_2 \oplus k) = m_1 \oplus m_2$$

→ No son iguales en general, pero la estructura XOR es explotable.

Otro ejemplo lineal: cifrado de Vernam, cifrado de Hill (si se ve como multiplicación matricial lineal).

Ejercicio 4 – PrivK^{CPA} con r determinístico (2 pts)

CONSIGNA

Dado el criptosistema $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$ para $p \in \mathbb{Z}$ y $a, b, r \in \mathbb{Z}_p$, clave $k = (a, b)$ y $r \leftarrow \text{random}$:

$$c = \text{Ek}(m) = (r, ar + b + m) \bmod p \quad | \quad m = \text{Dk}(c) = (-ar - b + c) \bmod p$$

Considerar variante donde $r = (a + b) \bmod p$ (no aleatorio). ¿Es seguro contra CPA?

Demostrar mediante $\text{PrivK}^{\text{CPA}}_{A, \Pi}$.

TIP

Si r es determinístico dado k , entonces Enc es determinístico $\rightarrow$ viola la condición mínima de IND-CPA. El atacante puede encriptar el mismo mensaje dos veces y verificar si el cifrado es igual.

RESOLUCIÓN

VARIANTE: $r = (a + b) \bmod p$ (fijo dado $k \rightarrow \text{Enc}$ es DETERMINÍSTICO)

EXPERIMENTO $\text{PrivK}^{\text{CPA}}(A, \Pi)$:

SETUP: $\text{Gen} \rightarrow k = (a, b)$ (secreto), $r = (a + b) \bmod p$ (fijo)

FASE 1 – El atacante A consulta el oráculo de encriptación:

A elige $m_{\text{test}} = 0$ y pide encriptar $\rightarrow$ obtiene:

$$c_{\text{test}} = (r, ar + b + 0 \bmod p) = (r, ar + b \bmod p)$$

FASE 2 – Challenge:

A elige $m_0 = 0$ y $m_1 = 1$ (distintos)

Challenger elige $b_{\text{chall}} \leftarrow_R \{0, 1\}$ y envía $c^* = \text{Ek}(m_{b_{\text{chall}}})$:

- Si $b = 0$: $c^* = (r, ar + b \bmod p)$ [= c_{test}]
- Si $b = 1$: $c^* = (r, ar + b + 1 \bmod p)$ [= $c_{\text{test}} + 1$ en segunda componente]

FASE 3 – El atacante A decide:

A compara c^* con c_{test} :

- Si segunda componente de $c^* == ar + b \bmod p \rightarrow A$ devuelve $b' = 0$
- Si segunda componente de $c^* == ar + b + 1 \bmod p \rightarrow A$ devuelve $b' = 1$

RESULTADO:

$$\Pr[\text{PrivK}^{\text{CPA}}(A, \Pi) = 1] = 1 \quad (\text{el atacante siempre gana})$$

$$\text{Ventaja} = |1 - 1/2| = 1/2$$

CONCLUSIÓN: El sistema NO es IND-CPA seguro. Al hacer r determinístico,

Enc es una función determinística de m (fijada k), violando el requisito de que esquemas IND-CPA deben ser probabilísticos.

La encriptación de $m=0$ siempre produce el mismo cifrado $\rightarrow$ trivialmente distinguible.

Ejercicio 5 – Verdadero o Falso (2 pts)

CONSIGNA

- a) Para proveer privacidad e integridad, lo correcto es: Cifrar m con k_1 para obtener c , obtener MAC con k_2 de m para obtener t . Transmitir m y t . Las claves k_1 y k_2 pueden ser iguales.
- b) La seguridad de las funciones de hash se establece como el nivel de resistencia a preimágenes donde dado un y hallar $x/h(x)=y$.
- c) El algoritmo de Diffie-Hellman permite que Alice le envíe una clave de sesión a Bob por un canal inseguro.
- d) En los cifrados en bloque independientemente del modo se requiere que BCE/PRF siempre sea reversible.

TIP

Encrypt-and-MAC: error de transmitir m en claro. Hash: tiene 3 propiedades, no solo preimagen. DH: acuerdo (no envío) de clave. CTR/OFB: no requieren invertir la primitiva.

RESOLUCIÓN

a) FALSO (múltiples errores)

Error 1: Se transmite m en claro junto con $t \rightarrow$ no hay confidencialidad.

Debe transmitirse el CIFRADO c , no m .

Error 2: El MAC debe aplicarse sobre c (Encrypt-then-MAC), no sobre m .

Error 3: k_1 y k_2 deben ser DISTINTAS (separación de claves).

CORRECCIÓN: "Cifrar m con k_1 para obtener c , obtener MAC con k_2 DE c para obtener t . Transmitir (c, t) . k_1 y k_2 DEBEN ser diferentes."

b) VERDADERO (pero incompleto – aceptar como verdadero)

La resistencia a preimagen es una de las tres propiedades de las funciones de hash criptográficas:

1. Resistencia a preimagen: dado $h(x)$, difícil encontrar x ✓
2. Resistencia a segunda preimagen: dado x_1 , difícil encontrar $x_2 \neq x_1$ con $h(x_1)=h(x_2)$
3. Resistencia a colisiones: difícil encontrar cualquier par (x_1, x_2) , $x_1 \neq x_2$ con $h(x_1)=h(x_2)$

La sentencia describe correctamente la resistencia a preimagen.

c) FALSO

DH es un protocolo de ACUERDO (agreement) de clave, no de ENVÍO.

Ninguna parte elige la clave y la "envía" a la otra. Ambas partes contribuyen con valores efímeros y derivan conjuntamente la misma clave.

CORRECCIÓN: "DH permite que Alice y Bob ACUERDEN/DERIVEN una clave de sesión compartida sobre un canal inseguro [sin que ninguno la envíe]."
[Cambio: "envíe una clave a Bob" → "acuerden una clave compartida"]

d) FALSO

En modos como CTR y OFB, la primitiva E_k se usa como generador de keystream:

- CTR: $\text{keystream} = E_k(\text{nonce} || \text{counter})$, luego $C_i = M_i \oplus \text{keystream}$
- OFB: $O_i = E_k(O_{i-1})$, luego $C_i = M_i \oplus O_i$

En ambos casos NUNCA se aplica D_k^{-1} . La PRF no necesita ser invertible. Solo en modos CBC y ECB se requiere D_k para descifrar.

CORRECCIÓN: "En algunos modos de operación (ECB, CBC) se requiere que la BCE sea reversible; en CTR y OFB NO es necesario."

[Cambio: "independientemente del modo" → especificar los modos]

Parcial 1 – 2018

Condición de aprobación: 5 puntos · 5 ejercicios de 2 pts c/u

Ejercicio 1 – Protocolo Needham-Schroeder con KDC (2 pts)

CONSIGNA

Dado el protocolo (T = Trusted Third Party / KDC):

(1.1) $A \rightarrow T: A, B, NA$ | (1.2) $T \rightarrow A: EkAT(NA, B, k, EkBT(k, A))$

(1.3) $A \rightarrow B: EkBT(k, A)$ | (1.4) $B \rightarrow A: Ek(NB)$ | (1.5) $A \rightarrow B: Ek(NB-1)$

- a) ¿Para qué está el nombre del destinatario en los mensajes (1.1) y (1.2)?
- b) ¿Qué problema tiene este protocolo?
- c) El protocolo de Denning-Sacco agrega timestamps a (1.2) y (1.3). ¿Con qué objetivo?

TIP

Needham-Schroeder clásico: el problema es el ataque de replay con clave comprometida. El nombre del destinatario previene sustitución de clave. Denning-Sacco agrega timestamps para frescura.

RESOLUCIÓN

a) PROPÓSITO DEL NOMBRE DEL DESTINATARIO:

En (1.1): A incluye B para indicar a T con quién quiere comunicarse, para que T genere una clave k para esa sesión específica A-B.

En (1.2): T incluye B DENTRO del cifrado $EkAT(NA, B, k, \dots)$.

Esto permite que A verifique que la clave k fue generada para hablar con B (y no con otra entidad). Sin este dato, un atacante podría reutilizar tickets de otras sesiones y A no podría distinguirlo.

b) PROBLEMA DEL PROTOCOLO – Ataque de Replay:

Si un atacante C obtuvo previamente una clave k de sesión (que ya expiró o fue comprometida), puede repetir el mensaje (1.3):

$C \rightarrow B: EkBT(k, A)$ [ticket antiguo]

B no puede verificar si k es fresca o antigua (no hay timestamp).

C puede hacerse pasar por A ante B reutilizando un ticket viejo.

Este ataque funciona aunque k sea una clave ya comprometida.

c) TIMESTAMPS EN DENNING-SACCO:

Denning-Sacco agrega timestamps T a los mensajes (1.2) y (1.3):

$EkAT(NA, B, k, T, EkBT(k, A, T))$

OBJETIVO: Garantizar FRESCURA de la clave k.

B verifica que el timestamp T sea reciente (dentro de una ventana de tiempo tolerada, ej: ± 5 minutos). Un replay de un ticket antiguo

(con T viejo) es detectado y rechazado automáticamente.

LIMITACIÓN: Requiere sincronización de relojes entre los participantes.
Si los relojes difieren, sesiones legítimas pueden ser rechazadas.

Ejercicio 2 – Opción múltiple (2 pts)

CONSIGNA

- (1) La validación de un Certificado Digital incluye: (a) verificar clave privada del cert vs clave pública del emisor; (b) verificar que la clave pública cifre la clave privada del emisor; (c) verificar que la firma de la CA sea válida.
- (2) Cifrado homofonico del Duque de Mantua (sustitución con múltiples símbolos por vocal, proporcional a frecuencia): (a) no tiene secreto perfecto porque no se puede identificar la vocal; (b) opera como un cifrado Vigenère; (c) el índice de coincidencia no es tan útil.
- (3) Confidencialidad e integridad: (a) SSL con KDC; (b) SSL con PKI da confidencialidad, integridad y no repudio; (c) TLS con PKI da confidencialidad, integridad y autenticación.

TIP

Certificado: validar = verificar firma de la CA. Homofonico equaliza frecuencias $\rightarrow IC \approx \text{texto random} \rightarrow IC$ inútil. TLS es el correcto (SSL deprecado, no KDC).

RESOLUCIÓN

(1) VALIDACIÓN DE CERTIFICADO DIGITAL:

$\rightarrow$ CORRECTA: (c) Verificar que la firma digital emitida por la CA incluida dentro del Certificado Digital sea válida.

Proceso: usar la clave pública de la CA (del certificado raíz, preinstalado)

para verificar la firma digital sobre el contenido del certificado.

(a) Incorrecto: los certificados no contienen claves privadas.

(b) Incorrecto: la clave pública no "cifra" la clave privada.

(2) CIFRADO HOMOFONICO:

$\rightarrow$ CORRECTA: (c) El índice de coincidencia no es tan útil en este caso.

Justificación: el cifrado homofonico asigna múltiples símbolos a cada vocal en proporción a su frecuencia de uso. Esto IGUALA la distribución de frecuencias del texto cifrado $\rightarrow IC$ del texto cifrado $\approx IC$ de texto aleatorio (≈ 0.038). El IC no puede distinguir el texto cifrado de texto aleatorio, haciéndolo inútil para criptoanálisis.

(a) Falso: el cifrado homofonico puede tener secreto perfecto si está bien diseñado. (b) Falso: no opera como Vigenère (no usa clave periódica).

(3) CONFIDENCIALIDAD E INTEGRIDAD:

$\rightarrow$ CORRECTA: (c) TLS ofrece confidencialidad, integridad y autenticación de los participantes bajo un esquema PKI de distribución de certificados.

- (a) SSL está deprecado; no usa KDC sino PKI.
- (b) TLS/SSL no proveen no repudio (usan MACs simétricos en la sesión).

Ejercicios 3, 4 y 5

NOTA

Los ejercicios 3 (CBC bloques eliminados), 4 ($\text{PrivK}^{\text{CPA}}$ con $r=(a+b)$) y 5 (Verdadero o Falso) son idénticos a los del Parcial 2023. Ver las soluciones completas en la sección correspondiente.

REFERENCIA

Ej. 3 – CBC (C_0 eliminado, C_n eliminado, prepending M_0):

Ver solución completa en Parcial 2023, Ejercicio 2. Respuestas idénticas.

Ej. 4 – $\text{PrivK}^{\text{CPA}}$ con $r=(a+b)$ determinístico:

Ver solución completa en Parcial 2023, Ejercicio 4. Respuestas idénticas.

Ej. 5 – Verdadero o Falso (Encrypt-and-MAC, preimagen, DH, PRF reversible):

Ver solución completa en Parcial 2023, Ejercicio 5. Respuestas idénticas.

Parcial 1 – 2017

Condición de aprobación: 5 puntos · 5 ejercicios (3+2+1,5+2+1,5 pts)

Ejercicio 1 – Cilindro de Jefferson (3 pts)

CONSIGNA

Cilindro de Jefferson: 36 discos, cada uno con 26 letras en una permutación particular. El mecanismo de encriptación alinea los 36 caracteres del mensaje en la ranura y elige una de las 25 alineaciones restantes como cifrado. Mensajes de 36 letras.

- a) ¿Cuál es el tamaño del espacio de claves?
- b) Demostrar si este sistema admite secreto perfecto y bajo qué condiciones.

TIP

La clave tiene dos componentes: orden de los discos (36!) y elección de alineación (25). Para secreto perfecto, comparar $|K|$ con $|M| = 26^{36}$. Con $|K| \ll |M| \rightarrow$ imposible tener secreto perfecto.

RESOLUCIÓN

a) ESPACIO DE CLAVES:

La clave tiene dos componentes independientes:

1. Permutación (orden) de los 36 discos: 36! formas posibles
2. Elección de una de las 25 alineaciones restantes para el cifrado: 25

$$|K| = 36! \times 25 \approx 3,7 \times 10^{41} \times 25 \approx 9,3 \times 10^{42}$$

b) ¿SECRETO PERFECTO?

Espacio de mensajes (36 letras de A-Z): $|M| = 26^{36} \approx 3,1 \times 10^{50}$

CONDICIÓN NECESARIA (Teorema de Shannon):

Para secreto perfecto se requiere $|K| \geq |M|$.

$$|K| = 9,3 \times 10^{42} < |M| = 3,1 \times 10^{50}$$

$\rightarrow |K| \ll |M| \rightarrow$ NO puede haber secreto perfecto.

DEMOSTRACIÓN FORMAL (por contraejemplo):

Fijado un mensaje m , hay exactamente 25 posibles cifrados c (las 25 alineaciones posibles, independientemente del orden de los discos – el orden define qué letras aparecen, pero dado m , solo hay 25 cifrados posibles).

Sea c un texto cifrado que NO puede provenir del mensaje m bajo ningún orden de discos ni alineación.

$$\rightarrow \Pr[C=c \mid M=m] = 0$$

Pero puede existir otro mensaje m' tal que $\Pr[C=c \mid M=m'] > 0$.

$$\rightarrow \Pr[C=c \mid M=m] \neq \Pr[C=c \mid M=m']$$

Esto viola la definición de secreto perfecto: $\Pr[C=c \mid M=m]$ debe ser igual para todo m .

CONCLUSIÓN: El sistema NO tiene secreto perfecto bajo ninguna condición, ya que $|K| < |M|$ es una barrera insuperable (Teorema de Shannon).

Ejercicio 2 – Shamir's No-Key Protocol (2 pts)

CONSIGNA

Protocolo de Shamir (Three-Pass): Gen: público p primo; A, B eligen a, b con $1 \leq a, b \leq p-2$, $a \perp (p-1)$, $b \perp (p-1)$. A elige K (clave de sesión).

Trans: (1) $A \rightarrow B: K^a \bmod p$ | (2) $B \rightarrow A: (K^a)^b \bmod p$ | (3) $A \rightarrow B: (K^{ab})^{a^{-1}} \bmod p = K^b \bmod p$

a) Detallar el objetivo y la seguridad computacional.

b) ¿En qué se diferencia con Diffie-Hellman?

TIP

En Shamir, A ELIGE la clave K y se la transfiere a B en 3 pasos. En DH, ambos contribuyen a derivar la clave. Ambos se basan en DLP.

RESOLUCIÓN

PROTOCOLO SHAMIR (Three-Pass):

(1) $A \rightarrow B: K^a \bmod p$

(2) $B \rightarrow A: K^{ab} \bmod p$

(3) $A \rightarrow B: K^{ab} \cdot a^{-1} = K^b \bmod p$

B calcula: $(K^b)^{b^{-1}} = K \quad \checkmark$

a) OBJETIVO Y SEGURIDAD:

OBJETIVO: A transfiere la clave secreta K a B por un canal INSEGURO sin haber compartido ningún secreto previamente (ni clave pública ni privada).

A diferencia de DH, aquí A ELIGE K y B la recibe al final.

SEGURIDAD COMPUTACIONAL:

Reside en el Problema del Logaritmo Discreto (DLP).

Un atacante ve: K^a , K^{ab} , K^b .

Para obtener K necesitaría:

- Calcular a a partir de $K^a \rightarrow \text{DLP}$
- Calcular b a partir de $K^{ab}/K^a \rightarrow \text{DLP}$

Para p suficientemente grande (≥ 2048 bits), esto es computacionalmente inviable con los algoritmos conocidos.

b) DIFERENCIAS CON DIFFIE-HELLMAN:

Aspecto	Shamir Three-Pass	Diffie-Hellman
# mensajes	3 (three-pass)	2
¿Quién elige K?	Solo A (activo)	Nadie: se deriva $g^{(xy)}$
Rol de B	Pasivo (recibe K)	Activo (contribuye con y)
g público	No es necesario	Sí, g y p son públicos
Resultado	B recibe exactamente K	Ambos derivan $g^{(xy)}$
Objetivo	Enviar clave de A a B	Acordar clave compartida

EN RESUMEN: DH permite ACORDAR una clave (ambos contribuyen).

Shamir permite TRANSFERIR una clave (A la elige, B la recibe).

Ejercicio 3 – Validación de certificado en browser (1,5 pts)

CONSIGNA

Especificar los pasos de validación que debería realizar un browser al conectarse a un homebanking para verificar que el certificado presentado por el sitio no es apócrifo.

TIP

Seguir la cadena de confianza: cert del sitio → CA intermedia → CA raíz (preinstalada). Verificar firma, vigencia, identidad (CN/SAN), y revocación (CRL/OCSP).

RESOLUCIÓN

PASOS DE VALIDACIÓN DEL BROWSER:

1. OBTENER EL CERTIFICADO:

El servidor presenta su certificado X.509 durante el TLS handshake.

2. VERIFICAR LA CADENA DE CONFIANZA (Chain of Trust):

- a) El certificado del sitio está firmado por una CA intermedia.
- b) El certificado de la CA intermedia está firmado por la CA raíz.
- c) La CA raíz debe estar en el almacén de CAs de confianza preinstalado (trusted root store) del sistema operativo o del browser.

Para cada nivel: verificar la firma digital con la clave pública del nivel superior.

3. VERIFICAR VIGENCIA (FECHAS):

Comprobar que la fecha actual esté entre "Not Before" y "Not After" de cada certificado en la cadena.

4. VERIFICAR IDENTIDAD (CN / SAN):

El Common Name (CN) o Subject Alternative Name (SAN) del certificado debe coincidir con el hostname visitado (ej: www.banco.com).

Protege contra certificados válidos emitidos para otro dominio.

5. VERIFICAR REVOCACIÓN:

- a) CRL (Certificate Revocation List): descargar la lista de certificados

revocados publicada por la CA y verificar que el cert no esté en ella.

- b) OCSP (Online Certificate Status Protocol): consultar en tiempo real el estado del certificado al servicio OCSP de la CA.

Si el cert está revocado → rechazar la conexión.

6. VERIFICAR USO DE CLAVE (Key Usage):

El campo Key Usage debe incluir usos apropiados para TLS
(Digital Signature, Key Encipherment).

7. RESULTADO:

Si todos los pasos son exitosos → mostrar candado / conexión segura.

Si alguno falla → advertencia al usuario o bloqueo de la conexión.

Ejercicio 4 – MAC-Forge con MAC de paridad (2 pts)

CONSIGNA

Dado el algoritmo de MAC:

$k \leftarrow \{0,1\} \mid \text{Mac}_k(m) = t$ donde t es el bit de paridad par si $k=0$, o impar si $k=1$

$\text{Vrfy}_k(m,t) = 1$ si ($k=0$ y t es paridad par de m) O ($k=1$ y t es paridad impar de m)

Demostrar mediante MAC-Forge cómo un atacante podría tener éxito.

TIP

El MAC revela k directamente: si el MAC de un mensaje con número impar de 1s es 0, entonces $k=0$ (paridad par). Con k conocido el atacante puede forjar cualquier MAC.

RESOLUCIÓN

ESQUEMA:

$k \leftarrow_R \{0,1\}$

$\text{Mac}_k(m): t=0$ si ($k=0$ y $\# \text{unos_en_m}$ es par) o ($k=1$ y $\# \text{unos_en_m}$ es impar)

$t=1$ si ($k=0$ y $\# \text{unos_en_m}$ es impar) o ($k=1$ y $\# \text{unos_en_m}$ es par)

EXPERIMENTO MAC-FORGE: $A(1^n)$

FASE 1 – Consulta al oráculo:

A elige $m_1 = "1000"$ ($\# \text{unos} = 1 \rightarrow$ impar)

A pide $\text{Mac}_k("1000") \rightarrow$ recibe t_1

Análisis:

- Si $k=0$: $t_1=1$ (paridad par requiere cambiar; "1000" es impar $\rightarrow t=1$ para hacerlo par)

Espera: t ES el bit de paridad, no el mensaje modificado.

$t=0$ significa que la cantidad de 1s ya es par ($k=0$).

$t=1$ significa que no lo es ($k=0$ pero la cantidad no es par).

Redefiniendo: $t=0 \leftrightarrow$ condición de paridad cumplida; $t=1 \leftrightarrow$ no cumplida.

Para $m_1="1000"$ ($\# \text{unos}=1$, impar):

- Si $k=0$ (paridad par): $\# \text{unos}$ es impar $\neq$ par $\rightarrow t_1=0$? No.

INTERPRETACIÓN CORRECTA: t es el bit que se agrega para completar la paridad.

Para "1000" (1 uno):

- $k=0$ (paridad par): agregar $t=1 \rightarrow "10001"$ tiene 2 unos (par) $\rightarrow t_1=1$

- $k=1$ (paridad impar): agregar $t=0 \rightarrow "10000"$ tiene 1 uno (impar) $\rightarrow t_1=0$

ESTRATEGIA DEL ATACANTE:

A consulta $\text{Mac}_k("1000") \rightarrow$ recibe t_1 .

- Si $t_1=1 \rightarrow k=0$ (necesitó agregar 1 para hacer paridad par)
 - Si $t_1=0 \rightarrow k=1$ (necesitó agregar 0 para hacer paridad impar)
- $\rightarrow$ A DESCUBRE k CON UNA SOLA CONSULTA.

FASE 2 – Forgery:

A elige $m^* = "0000"$ (#unos=0, par). $m^* \neq m_1$.

Con k conocido:

- Si $k=0$: $\text{Mac}_k("0000")=0$ (0 unos es par $\rightarrow$ agregar 0 para mantener par)
- Si $k=1$: $\text{Mac}_k("0000")=1$ (0 unos es par $\rightarrow$ agregar 1 para hacer impar)

A presenta $(m^*="0000", t^*)$ según el k que descubrió.

$\text{Vrfy}_k(m^*, t^*) = 1 \checkmark$

RESULTADO: MAC-Forge exitoso con 1 consulta, $\text{Pr}=1$.

CONCLUSIÓN: El MAC no es seguro. La observación de t revela k directamente.

Ejercicio 5 – Opción múltiple (1,5 pts)

CONSIGNA

- (1) En el modo OFB: (a) C_i surge de $Enc(P_i)$; (b) error en C_i afecta todo desde ese punto; (c) O_j se genera encriptando O_{j-1} ; (d) XOR inicial sobre IV que se encripta formando K_i .
- (2) Para validar CVV en servidores del banco sin tenerlo en claro en BD: (a) HMAC con clave simétrica; (b) AES-256 con clave pública; (c) Diffie-Hellman; (d) Firma digital.
- (3) Para distribución de software en mercados de apps: (a) TDES; (b) CBC-MAC con PRF; (c) Firma digital sobre binarios; (d) SHA-1 sobre ejecutables.

TIP

OFB: keystream = cadena de salidas del cipher. CVV: necesita ser unidireccional → HMAC.
Software: necesita autenticidad de origen → firma digital con clave privada del desarrollador.

RESOLUCIÓN

(1) MODO OFB:

→ CORRECTA: (c) La salida O_j de cada bloque de encriptación se genera encriptando la salida del bloque O_{j-1} .

En OFB: $O_0 = Ek(IV)$, $O_1 = Ek(O_0)$, $O_2 = Ek(O_1)$, ..., $C_i = M_i \oplus O_i$

El keystream es independiente del plaintext y ciphertext.

- (a) Falso: eso es ECB.
- (b) Falso: en OFB un error en C_i solo afecta M_i (no se propaga); la propagación total ocurre en CBC.
- (d) Falso: en OFB el IV se encripta directamente para generar O_0 , sin XOR previo.

(2) VALIDACIÓN DE CVV:

→ CORRECTA: (a) Un HMAC basado en una clave simétrica almacenada de manera segura en el banco emisor.

El banco almacena HMAC(CVV) en la BD (no el CVV en claro). Para validar: recomputa HMAC(CVV_recibido) y compara. El HMAC es unidireccional → no se puede recuperar el CVV a partir del hash almacenado.

- (b) AES con clave pública es una contradicción (AES es simétrico).
- (c) DH es intercambio de claves, no validación de datos.
- (d) Firma digital para verificar CVV expondría el CVV en el mensaje firmado.

(3) DISTRIBUCIÓN DE SOFTWARE:

→ CORRECTA: (c) Un esquema de firma digital sobre los binarios.

La firma digital garantiza:

- AUTENTICIDAD: el binario viene del desarrollador genuino (solo él tiene kpr)
- INTEGRIDAD: el binario no fue modificado (cualquier cambio invalida la firma)

El usuario verifica con la clave pública del desarrollador.

- (a) TDES simétrico: no puede verificar autenticidad sin compartir clave.
- (b) CBC-MAC: sin clave pública no verifica autoría del desarrollador.
- (d) SHA-1 sin clave: cualquiera puede recalcular el hash → no autentica.

Parcial 1 – 2016

Condición de aprobación: 5 puntos · 5 ejercicios (3+2+1,5+2+1,5 pts)

Ejercicio 1 – Transposición por columnas: secreto perfecto (3 pts)

CONSIGNA

Esquema de transposición por columnas: Gen elige $k \in \{2, 3\}$ uniformemente. Mensajes de 6 letras distribuidos en k columnas, cifrado = lectura de filas de la transpuesta. Demostrar formalmente si tiene secreto perfecto para:

- a) $M = S^6$ donde $S = \{a, b, c, d, e, f\}$ (mensajes de 6 letras del alfabeto S)
- b) $M \subseteq S^6$ donde todos los m_i son distintos (permutaciones de $\{a, b, c, d, e, f\}$)

TIP

$|K|=2$. Condición necesaria de Shannon: $|K| \geq |M|$. En (a) $|M|=6^6=46656 \gg 2$. En (b) $|M|=6!=720 \gg 2$. En ambos casos $|K| < |M| \rightarrow$ imposible secreto perfecto. Dar también un contraejemplo explícito.

RESOLUCIÓN

SISTEMA: $k \in \{2, 3\}$, $\Pr[k=2]=\Pr[k=3]=1/2$, mensajes de 6 letras.

Con $k=2$: distribuir en 2 col $\times$ 3 filas, leer transpuesta (3 col $\times$ 2 filas)

Con $k=3$: distribuir en 3 col $\times$ 2 filas, leer transpuesta (2 col $\times$ 3 filas)

$$|K| = 2$$

$$\text{a) } M = S^6, |M| = 6^6 = 46.656$$

POR TEOREMA DE SHANNON (condición necesaria):

Para secreto perfecto se requiere $|K| \geq |M|$.

$$|K| = 2 < 46.656 = |M| \rightarrow \text{NO puede haber secreto perfecto. } \times$$

CONTRAJEJEMPLO EXPLÍCITO:

Definición usada: $\Pr[C=c \mid M=m] = \Pr[C=c]$ para todo m, c .

Fijado cualquier mensaje m , solo existen 2 posibles cifrados (uno para $k=2$ y otro para $k=3$).

$$\rightarrow \Pr[C=c \mid M=m] \in \{0, 1/2\} \text{ para cualquier } c.$$

Existe c tal que $\Pr[C=c] > 0$ (algún m' lo produce) pero $\Pr[C=c \mid M=m] = 0$.

$$\rightarrow \Pr[C=c \mid M=m] \neq \Pr[C=c] \rightarrow \text{NO hay secreto perfecto. } \times$$

$$\text{b) } M = \{\text{permutaciones de } \{a, b, c, d, e, f\}\}, |M| = 6! = 720$$

POR TEOREMA DE SHANNON:

$|K| = 2 < 720 = |M| \rightarrow$ NO puede haber secreto perfecto. ✗

CONTRA EJEMPLO:

$m = \text{"abcdef"}$ con $k=2$:

Col: a c e $\rightarrow$ traspuesta: a d, c e, e f $\rightarrow c = \text{"adcebf"}$?

(Distribución: fila1=a,c,e; fila2=b,d,f $\rightarrow$ tras: col1=a,b; col2=c,d;

col3=e,f

$\rightarrow$ leyendo por filas: "abcdef" – ¡el cifrado coincide con el original para $k=2$!)

Con $k=3$: fila1=a,b,c; fila2=d,e,f $\rightarrow$ tras: col1=a,d; col2=b,e; col3=c,f

$\rightarrow c = \text{"adbecf"}$

Ahora: $\Pr[C=\text{"adbecf"} \mid M=\text{"abcdef"}] = 1/2$ (solo ocurre con $k=3$)

$\Pr[C=\text{"adbecf"}] < 1/2$ (otros mensajes pueden dar el mismo cifrado con $k=3$)

$\rightarrow$ La igualdad no se mantiene para todos los $m \rightarrow$ NO hay secreto perfecto.

✗

CONCLUSIÓN: En ninguno de los dos casos hay secreto perfecto.

Causa raíz: $|K|=2$ es insuficiente para $|M| \gg 2$.

Ejercicio 2 – Protocolos con hash y firma (2 pts)

CONSIGNA

Protocolo 1: $y = \text{Enc}_{k1}(x \parallel H(k2 \parallel x))$ — x mensaje, H hash, Enc simétrico, $k1$ y $k2$ claves secretas compartidas.

Protocolo 2: $y = \text{Enc}_k(x \parallel \text{Sign}_{kprA}(H(x)))$ — k clave simétrica compartida, $kprA$ clave privada de A .

- a) Explicar qué pasos efectúa B al recibir y en cada protocolo.
 b) Para cada protocolo: ¿se preservan confidencialidad, integridad, no repudio, seguridad ante replay?

TIP

P1: MAC simétrico = autenticación entre partes que comparten $k2$, sin no repudio. P2: firma asimétrica = no repudio. Ninguno tiene protección anti-replay (no hay nonces ni timestamps).

RESOLUCIÓN

a) PASOS DE B:

PROTOCOLO 1: $y = \text{Enc}_{k1}(x \parallel H(k2 \parallel x))$

1. Descifrar: $(x', h') = \text{Dec}_{k1}(y)$ usando $k1$
2. Recomputar: $h_check = H(k2 \parallel x')$ usando $k2$
3. Verificar: ¿ $h' == h_check$?
 - Sí → mensaje íntegro y auténtico → aceptar x'
 - No → rechazar (alterado o forjado)

PROTOCOLO 2: $y = \text{Enc}_k(x \parallel \text{Sign}_{kprA}(H(x)))$

1. Descifrar: $(x', s') = \text{Dec}_k(y)$ usando k (clave simétrica)
2. Calcular hash: $h = H(x')$
3. Verificar firma: $\text{Vrfy}_{kpubA}(h, s') = 1?$ usando clave pública de A
 - Sí → autenticidad e integridad garantizadas → aceptar x'
 - No → rechazar

b) PROPIEDADES:

Propiedad	Protocolo 1	Protocolo 2

CONFIDENCIALIDAD	SÍ: Enc _{k1} cifra todo	SÍ: Enc _k cifra todo
INTEGRIDAD	SÍ: $H(k2 x)$ es MAC	SÍ: $\text{Sign}_{\text{kprA}}(H(x))$
	detecta modificaciones	detecta modificaciones
NO REPUDIO	NO: k2 es compartida;	SÍ: Solo A tiene kprA;
	B también puede generar	B puede probar ante
	el MAC → no puede	terceros que el
mensaje	probar autoría de A	vino de A
ANTI-REPLAY	NO: sin nonces ni	NO: sin nonces ni
	timestamps → un msg	timestamps → ídem
	puede reenviarse	

Ejercicio 3 – Modo FSM (Fischer Spiffy Mixer) (1,5 pts)

CONSIGNA

Modo FSM: $c_0 = IV_2$, $m_0 = IV_1$, $c_i = F_k(m_{i-1} \oplus c_{i-1}) \oplus m_i$ para $i \geq 1$.

- Describir cómo se hace la descricpción.
- Si un bit en el bloque cifrado c_j se daña, ¿qué bloques de texto se descifrarán mal?
¿Por qué?

TIP

Para descifrar: despejar m_i de $c_i = F_k(m_{i-1} \oplus c_{i-1}) \oplus m_i \rightarrow m_i = c_i \oplus F_k(m_{i-1} \oplus c_{i-1})$. El error en c_j afecta m_j y m_{j+1} y todos los siguientes (propagación infinita).

RESOLUCIÓN

MODO FSM: $c_i = F_k(m_{i-1} \oplus c_{i-1}) \oplus m_i$

a) DESCIFRADO:

Despejando m_i de la ecuación de cifrado:

$$m_i = c_i \oplus F_k(m_{i-1} \oplus c_{i-1})$$

Proceso secuencial (requiere conocer m_{i-1} y c_{i-1} anteriores):

- $m_0 = IV_1$ (conocido)
- $m_1 = c_1 \oplus F_k(m_0 \oplus c_0) = c_1 \oplus F_k(IV_1 \oplus IV_2)$
- $m_2 = c_2 \oplus F_k(m_1 \oplus c_1)$
- $m_3 = c_3 \oplus F_k(m_2 \oplus c_2)$
- ...
- $m_i = c_i \oplus F_k(m_{i-1} \oplus c_{i-1})$

b) PROPAGACIÓN DE ERROR EN c_j :

Si c_j tiene un bit dañado (c_j^*):

- $m_j = c_j^* \oplus F_k(m_{j-1} \oplus c_{j-1})$
→ INCORRECTO: usa c_j^* dañado. m_j queda completamente corrupto.
- $m_{j+1} = c_{j+1} \oplus F_k(m_j \oplus c_j^*)$
→ INCORRECTO: usa m_j corrupto Y c_j^* dañado. Completamente corrupto.
- $m_{j+2} = c_{j+2} \oplus F_k(m_{j+1} \oplus c_{j+1})$
→ INCORRECTO: usa m_{j+1} corrupto. c_{j+1} es correcto pero m_{j+1} no.
- $m_{j+3}, m_{j+4}, \dots$: siguen siendo incorrectos porque cada m_i depende del m_{i-1} anterior, que sigue estando corrupto.

El error NO se recupera nunca: la corrupción se propaga INDEFINIDAMENTE a todos los bloques desde mj en adelante.

CONCLUSIÓN: FSM tiene la PEOR tolerancia a errores de todos los modos. La doble retroalimentación (ci y mi) hace que cualquier error en ci se propague de forma irreversible a todo el resto del mensaje.

Ejercicio 4 – PKI vs KDC (2 pts)

CONSIGNA

- Explicar qué significan las siglas PKI y KDC.
- Comparar PKI y KDC (al menos una similitud y una diferencia).

RESOLUCIÓN

a) SIGLAS:

PKI = Public Key Infrastructure (Infraestructura de Clave Pública)
Sistema de gestión de certificados digitales que permite distribuir y validar claves públicas mediante una jerarquía de CAs (Certification Authorities). Ejemplos: HTTPS, firma digital de documentos.

KDC = Key Distribution Center (Centro de Distribución de Claves)
Servidor centralizado de confianza que distribuye claves de sesión simétricas a los participantes que desean comunicarse de forma segura.

Cada usuario comparte una clave con el KDC. Ejemplo: Kerberos.

b) COMPARACIÓN:

SIMILITUD:

Ambos son Trusted Third Parties (TTP): actúan como árbitros de confianza que facilitan el establecimiento de comunicación segura entre partes que no se conocen previamente y no comparten secretos iniciales.

DIFERENCIAS:

Aspecto	PKI	KDC
Tipo de clave	Pública/asimétrica	Simétrica
Requiere online?	No (verificación offline)	Sí (debe estar online)
Escalabilidad	Alta (cert. independiente)	Limitada (SPOF)

Riesgo central	Bajo (CA solo firma)	Alto (KDC conoce
claves)		
Mecanismo	Certificados X.509	Tickets/claves sesión
Ejemplo	TLS/HTTPS, S/MIME	Kerberos, N-S protocol

Ejercicio 5 – Opción múltiple (1,5 pts)

CONSIGNA

- (1) Protocolo DH descripto (Alice elige $x \rightarrow h1=g^x$, Bob elige $y \rightarrow h2=g^y$, $kA=h2^x$, $kB=h1^y$): es protocolo de: (a) MAC; (b) firma asimétrica; (c) intercambio de claves; (d) encriptación asimétrica.
- (2) Criptosistema CPA-seguro con PRF F_k : (a) $c=F_k(m) \oplus k$; (b) $c=r, m \oplus r$; (c) $c=r, F_k(r) \oplus m$; (d) $c=F_k(m)$.
- (3) CBC-MAC es seguro: (a) mensajes longitud variable + IV aleatorio; (b) mensajes longitud variable + IV constante; (c) mensajes longitud fija + IV constante; (d) mensajes longitud variable + IV constante.

RESOLUCIÓN

(1) PROTOCOLO:

→ CORRECTA: (c) Es un protocolo de intercambio de claves (Diffie-Hellman). Alice y Bob derivan la misma clave $g^{(xy)}$ sin transmitirla explícitamente. No es MAC (no hay autenticación de mensajes). No es firma (no hay documento). No es encriptación asimétrica (no hay cifrado de datos).

(2) CPA-SEGURO CON PRF:

→ CORRECTA: (c) $c = (r, F_k(r) \oplus m)$, $r \leftarrow_R \{0,1\}^n$

Análisis:

(a) $c=F_k(m) \oplus k$: determinístico dado $m, k \rightarrow$ NO CPA (mismo input = mismo output)

(b) $c=r, m \oplus r$: aleatorizado pero XOR con r puro no usa F_k como PRP $\rightarrow$ inseguro

(c) $c=r, F_k(r) \oplus m$: r es nonce aleatorio, $F_k(r)$ actúa como pad OTP $\rightarrow$ IND-CPA ✓

Si F_k es PRF, $F_k(r)$ es indistinguible de aleatorio $\rightarrow m$ queda oculto.

(d) $c=F_k(m)$: determinístico $\rightarrow$ NO CPA

(3) CBC-MAC SEGURO:

→ CORRECTA: (c) Si los mensajes son de longitud FIJA y el vector IV es CONSTANTE ($=0$).

(a) Longitud variable + IV aleatorio: inseguro (si IV es libre, el atacante

puede manipularlo para crear forgeries en mensajes de longitud variable)

(b) Longitud variable + IV constante: inseguro (ataque de extensión de longitud:

dado $MAC(m)$, un atacante puede calcular $MAC(m|m')$ sin conocer k)

(c) Longitud fija + IV constante ($\neq 0$): SEGURO ✓ – CBC-MAC es PRF para mensajes

de longitud fija, demostrado formalmente

(d) igual que (b)

Parcial 1 – 2015

Condición de aprobación: 5 puntos · 5 ejercicios (3+1,5+2+1,5+2 pts)

Ejercicio 1 – Cifrado Hill en Z_2 (3 pts)

CONSIGNA

Cifrado Hill con $M=S^2$ ($S=\{a,b\}$), claves = matrices 2×2 invertibles sobre Z_2 (coeficientes en $\{0,1\}$). Enc: $[c_1, c_2] = [m_1, m_2] \times k \bmod 2$. Dec: $[m_1, m_2] = [c_1, c_2] \times k^{-1} \bmod 2$. ($a=0, b=1$)

- ¿Por qué hay que descartar matrices con $\det=0$?
- Calcular $\#M$, escribir la tabla de encriptación y calcular $\#C$.
- Demostrar formalmente si tiene secreto perfecto.

TIP

En Z_2 , $\det=0 \rightarrow$ no invertible $\rightarrow$ Dec indefinido. Las 16 matrices 2×2 sobre Z_2 : solo 6 tienen $\det \neq 0 \bmod 2$. Para secreto perfecto: verificar $\Pr[C=00|M=00]=1 \neq \Pr[C=00|M=01]=0 \rightarrow$ viola independencia.

RESOLUCIÓN

- a) ¿POR QUÉ DESCARTAR MATRICES CON $\det(k)=0$?

Si $\det(k)=0 \bmod 2$, la matriz k no es invertible sobre Z_2 .

$\rightarrow$ No existe $k^{-1} \rightarrow \text{Dec}(c) = c \times k^{-1}$ es indefinido $\rightarrow$ imposible descifrar.

Además, si $\det(k)=0$: la función Enc_k no es inyectiva (dos mensajes distintos pueden dar el mismo cifrado $\rightarrow$ ambigüedad en el descifrado).

Para que el sistema sea correcto ($\text{Dec}(\text{Enc}(m))=m$), k debe ser invertible.

- b) ESPACIOS:

$M = \{aa=00, ab=01, ba=10, bb=11\} \rightarrow \#M = 4$

$C = \{00, 01, 10, 11\} \rightarrow \#C = 4$

Las 6 matrices invertibles sobre Z_2 son:

$k_1 = [1, 0; 0, 1] \quad k_2 = [1, 1; 0, 1] \quad k_3 = [1, 0; 1, 1]$

$k_4 = [0, 1; 1, 0] \quad k_5 = [1, 1; 1, 0] \quad k_6 = [0, 1; 1, 1]$

TABLA DE ENCRIPCIÓN $[m_1, m_2] \times k \bmod 2$:

$m \setminus k$	k_1	k_2	k_3	k_4	k_5	k_6
00	00	00	00	00	00	00
01	01	01	11	10	10	11
10	10	11	10	01	11	01
11	11	10	01	11	01	10

- c) SECRETO PERFECTO:

Definición: $\Pr[C=c \mid M=m] = \Pr[C=c]$ para todo m, c .

Observar que para $m=00$: $\text{Enc}(00, k) = 00$ para TODA k .

$$\rightarrow \Pr[C=00 \mid M=00] = 1$$

Para $m=01$: según la tabla, $01 \rightarrow 01(k1), 01(k2), 11(k3), 10(k4), 10(k5), 11(k6)$

$$\rightarrow \Pr[C=00 \mid M=01] = 0$$

Como $\Pr[C=00 \mid M=00] = 1 \neq 0 = \Pr[C=00 \mid M=01]$:

$$\rightarrow \Pr[C=00 \mid M=00] \neq \Pr[C=00 \mid M=01]$$

Esto viola la definición de secreto perfecto.

CONCLUSIÓN: El sistema NO tiene secreto perfecto.

La causa raíz es que $m=00$ (vector nulo) siempre cifra a 00 sin importar la clave, revelando información sobre el plaintext.

Ejercicio 2 – MAC no seguro: MAC-Forge (1,5 pts)

CONSIGNA

Esquema MAC para mensajes de longitud $2n$: $\text{Mac}_k(m_1||m_2) = (\text{Fk}(m_1), \text{Fk}(\text{Fk}(m_2))) = (t_1, t_2)$. Vrfy: verificar $t_1 = \text{Fk}(m_1)$ y $t_2 = \text{Fk}(\text{Fk}(m_2))$. Mostrar que no es seguro mediante el experimento MAC-Forge.

TIP

Ataque de cross-message forgery: consultar $\text{MAC}(m_1||m_2)$ y $\text{MAC}(m_3||m_4)$. Forjar $\text{MAC}(m_1||m_4) = (t_1_de_m_1, t_2_de_m_4)$. El verificador acepta porque $t_1 = \text{Fk}(m_1) \checkmark$ y $t_2 = \text{Fk}(\text{Fk}(m_4)) \checkmark$.

RESOLUCIÓN

ESQUEMA: $\text{Mac}_k(m_1||m_2) = (\text{Fk}(m_1), \text{Fk}(\text{Fk}(m_2)))$

$\text{Vrfy}_k(m, (t_1, t_2)) = 1$ sii $t_1 = \text{Fk}(m_1)$ y $t_2 = \text{Fk}(\text{Fk}(m_2))$

EXPERIMENTO MAC-Forge:

FASE 1 – Consultas al oráculo:

Consulta 1: A elige $m^{(1)} = m_1||m_2 \rightarrow$ recibe $t^{(1)} = (\text{Fk}(m_1), \text{Fk}(\text{Fk}(m_2)))$

Consulta 2: A elige $m^{(2)} = m_3||m_4 \rightarrow$ recibe $t^{(2)} = (\text{Fk}(m_3), \text{Fk}(\text{Fk}(m_4)))$

donde $m_1 \neq m_3$ y $m_2 \neq m_4$ (cuatro bloques distintos).

FASE 2 – Forgery:

A construye $m^* = m_1||m_4$ (primer bloque de $m^{(1)}$, segundo de $m^{(2)}$)

A construye $t^* = (t^{(1)}_1, t^{(2)}_2) = (\text{Fk}(m_1), \text{Fk}(\text{Fk}(m_4)))$

Nótese que $m^* = m_1||m_4$ NO fue consultado al oráculo.

VERIFICACIÓN:

$\text{Vrfy}_k(m^*, t^*)$:

- ¿ $t^*_1 == \text{Fk}(m_1)$? $\rightarrow \text{Fk}(m_1) == \text{Fk}(m_1) \checkmark$ (tomado de $t^{(1)}$)

- ¿ $t^*_2 == \text{Fk}(\text{Fk}(m_4))$? $\rightarrow \text{Fk}(\text{Fk}(m_4)) == \text{Fk}(\text{Fk}(m_4)) \checkmark$ (tomado de $t^{(2)}$)

$\rightarrow \text{Vrfy}$ devuelve 1 $\checkmark$

RESULTADO: MAC-Forge exitoso con 2 consultas.

$\Pr[\text{MAC-Forge}(A, \Pi) = 1] = 1$ (ventaja = 1)

CONCLUSIÓN: El esquema NO es un MAC seguro. El problema es que los dos componentes del tag son verificados independientemente, permitiendo

combinar componentes de tags de distintos mensajes para forjar un nuevo tag válido para un mensaje no consultado.

Ejercicio 3 – Propagating CBC (2 pts)

CONSIGNA

Modo PCBC: $c_0 = m_0 \oplus IV$, $c_i = F_k(m_i \oplus m_{i-1} \oplus c_{i-1})$ para $i \geq 1$.

- Describir cómo se hace la descricpción.
- Un error en un bit de c_j , ¿se propaga? ¿Por qué?

RESOLUCIÓN

MODO PCBC: $c_0 = m_0 \oplus IV$, $c_i = F_k(m_i \oplus m_{i-1} \oplus c_{i-1})$ para $i \geq 1$.

a) DESCIFRADO:

De $c_i = F_k(m_i \oplus m_{i-1} \oplus c_{i-1})$:

$\rightarrow F_k^{-1}(c_i) = m_i \oplus m_{i-1} \oplus c_{i-1}$

$\rightarrow m_i = F_k^{-1}(c_i) \oplus c_{i-1} \oplus m_{i-1}$

Para c_0 : $m_0 = c_0 \oplus IV$

Proceso secuencial:

- $m_0 = c_0 \oplus IV$
- $m_1 = F_k^{-1}(c_1) \oplus c_0 \oplus m_0$
- $m_2 = F_k^{-1}(c_2) \oplus c_1 \oplus m_1$
- $m_i = F_k^{-1}(c_i) \oplus c_{i-1} \oplus m_{i-1}$

b) PROPAGACIÓN DE ERROR EN c_j (1 bit dañado, c_j^*):

- $m_j = F_k^{-1}(c_j^*) \oplus c_{j-1} \oplus m_{j-1}$
 $\rightarrow$ INCORRECTO: usa c_j^* dañado $\rightarrow m_j$ completamente corrupto.
- $m_{j+1} = F_k^{-1}(c_{j+1}) \oplus c_j^* \oplus m_j$
 $\rightarrow$ INCORRECTO: usa c_j^* Y m_j dañados $\rightarrow$ completamente corrupto.
- $m_{j+2} = F_k^{-1}(c_{j+2}) \oplus c_{j+1} \oplus m_{j+1}$
 $\rightarrow$ INCORRECTO: m_{j+1} corrupto $\rightarrow m_{j+2}$ corrupto.

Sí, el error SE PROPAGA INDEFINIDAMENTE.

¿Por qué? Porque m_i depende de m_{i-1} (retroalimentación del plaintext).

Un m_i corrupto "contamina" todos los bloques siguientes. A diferencia de CBC estándar (donde c_i incorrecto solo afecta m_i y 1 bit de m_{i+1}), en PCBC la retroalimentación de plaintext propaga el error sin límite.

Ejercicio 4 – Opción múltiple (1,5 pts)

CONSIGNA

(1) En esquema de clave pública: (a) pública secreta, privada distribuida; (b) privada secreta, pública distribuida autenticada; (c) ambas secretas; (d) privada secreta, pública en KDC.

(2) Firma RSA con hash: $s = [H(m)]^d \bmod N$, $(N, d) = sk$. Para verificar Vrfy debe: (a) $s^e \bmod N$ y $H(s^e \bmod N)$; (b) $s^e \bmod N$ y $H^{-1}(s^e \bmod N)$; (c) $s^e \bmod N$ y $H(m)$; (d) $m^e \bmod N$ y $H(s)$.

RESOLUCIÓN

(1) ESQUEMA DE CLAVE PÚBLICA:

→ CORRECTA: (b) La clave privada se mantiene en secreto y la clave pública se distribuye autenticada (a través de certificados digitales).

(a) Invertido: lo público no es secreto.

(c) Si ambas fueran secretas sería un esquema simétrico, no asimétrico.

(d) La clave pública no se mantiene en un KDC; eso sería para claves simétricas.

(2) VERIFICACIÓN DE FIRMA RSA CON HASH:

Firma: $s = [H(m)]^d \bmod N$ (usando clave privada d)

Verificación: $s^e \bmod N$ debe dar $H(m)$ (usando clave pública e , N)

→ CORRECTA: (c) Vrfy debe recibir m y s , y efectuar:

- Calcular $s^e \bmod N$ (desencriptar la firma)

- Calcular $H(m)$ (hash del mensaje recibido)

- Verificar: $s^e \bmod N == H(m)$ ✓

(a) Verificar $H(s^e \bmod N)$ no tiene sentido (se hasha el resultado).

(b) H^{-1} no existe (hash es unidireccional).

(d) $m^e \bmod N$ encripta m con clave pública, no es parte de la verificación.

Ejercicio 5 – Ataque de suplantación en protocolo de autenticación (2 pts)

CONSIGNA

Protocolo de autenticación mutua:

- (1) $A \rightarrow B: nA$
- (2) $B \rightarrow A: \text{CertB}, \text{Sign_kB}(nA, nB)$
- (3) $A \rightarrow B: \text{CertA}, \text{Sign_kA}(nB, nA)$

Mostrar cómo un adversario C puede autenticarse como A ante B.

TIP

Ataque de sesión paralela: C abre dos sesiones simultáneas. Usa la respuesta de A en una sesión para autenticarse como A en la otra. La vulnerabilidad: $\text{Sign_kA}(nB, nA) \rightarrow$ C consigue que A firme nB enviándoselo como desafío.

RESOLUCIÓN

ATAQUE DE SUPLANTACIÓN: C se autentica como A ante B.

C no posee la clave privada de A (kA), pero puede usar un ataque de SESIÓN PARALELA:

SESIÓN 1 (C hace pasar por A ante B):

- (1a) $C \rightarrow B: nC$ [C envía nonce propio, iniciando sesión como "A"]
- (2a) $B \rightarrow C: \text{CertB}, \text{Sign_kB}(nC, nB)$
[B responde con su firma sobre (nC, nB)]
C necesita responder con $\text{CertA}, \text{Sign_kA}(nB, nC) \rightarrow$ ¡no tiene kA !

SESIÓN 2 PARALELA (C inicia sesión con A):

- (1b) $C \rightarrow A: nB$ [C envía nB (el nonce de B) como si fuera su desafío]
- (2b) $A \rightarrow C: \text{CertA}, \text{Sign_kA}(nB, nA2)$
[A firma (nB, nA2) con su clave privada $kA \rightarrow$ C obtiene lo que necesita]

C COMPLETA LA SESIÓN 1:

- (3a) $C \rightarrow B: \text{CertA}, \text{Sign_kA}(nB, nA2)$

B verifica:

- CertA válido ✓ (es el certificado real de A)
- $\text{Sign_kA}(nB, nC)$? Espera $\text{Sign_kA}(nB, nC)$ pero recibe $\text{Sign_kA}(nB, nA2) \dots$

NOTA: El ataque funciona si $nA2=nC$ (poco probable) O si el protocolo solo verifica el primer elemento (nB) de la firma.

VERSIÓN CORRECTA DEL ATAQUE (más precisa):

Si el protocolo acepta $\text{Sign_kA}(nB, *)$ sin verificar el segundo elemento:
C reenvía $\text{Sign_kA}(nB, nA2)$ y B lo acepta $\rightarrow$ C autenticado como A. ✓

SOLUCIÓN: incluir la identidad del destinatario y del sesión en la firma:
 $\text{Sign_kA}(B, nB, nA)$ en lugar de $\text{Sign_kA}(nB, nA)$, vinculando la firma a la identidad de B y evitando reutilización entre sesiones.

Criptografía y Seguridad 72.44 · Resoluciones para estudio académico · 2015–2026

Para guardar como PDF: presionar el botón "Guardar como PDF" $\rightarrow$ Ctrl+P $\rightarrow$ Guardar como PDF (activar "Gráficos de fondo" en Más opciones)